SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — a role-aware 4D view of a moving scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *linear-motion grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = the target's SE(3) forecast at the execution times of the chunk, taken from the training-time heads evaluated offline; light orange arrows = sparse 3D flow on the context on the 14×14 grid only; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ execution τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents at uniform fidelity" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, sparse 3D flow for the context, training only" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix E.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **A role-aware 4D view of a moving scene.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose precisely and the surrounding scene as a sparse 3D flow (orange), a forecast it skips at deployment, so that its commands meet the target when they execute (purple). Image-space world–action models forecast appearance at uniform fidelity instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires actions that fit the scene at the moment they execute, not when the image was captured. A policy must therefore infer how the scene moves and anticipate where it will be, yet every millisecond spent on this inference lets the target move further. Existing methods resolve this tension poorly: world–action models forecast future images or latents at uniform fidelity, spending capacity and time on appearance while leaving the target's metric pose implicit, and target-centric predictors ignore the surroundings that constrain the motion. We observe that the parts of a dynamic scene differ in the precision they require: the target's pose determines contact to within millimeters, whereas the rest of the scene matters only through the clearance it leaves. We present SE(3)-WAM, a world–action model that learns this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history and is trained to forecast the target's future SE(3) trajectory precisely and the surrounding scene coarsely as a sparse 3D flow. Its action stream reads the representation shaped by these forecasts and generates action chunks by conditional flow matching, replanning after every step; the forecasts themselves are never fed back to the actions and are skipped at deployment, so inference needs neither past frames nor a generated future and stays fast enough for real-time control. On Dynamic-14, a simulated benchmark in which the scene keeps moving during inference, SE(3)-WAM achieves [XX.X]% success, [XX.X] points above the strongest baseline, and it succeeds in [XX]% of trials across five real-robot tasks with linear, circular, and accelerating target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [9, 16, 37, 57, 76, 85, 107], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. This poses two coupled difficulties. A single image does not reveal how the target is moving, and by the time an action executes, the target is no longer where it was seen: a part on a conveyor at 0.1 m/s travels 15 mm during a 150 ms delay, more than a parallel-jaw grasp typically tolerates (Fig. 1).

Recent work equips policies with a memory of past motion and a forecast of the future [19, 50, 68, 95]. Both are costly in the resource that dynamic manipulation lacks most: time. Richer histories and denser forecasts add computation, and computation adds latency, which enlarges the very error the forecast is meant to remove. Existing designs therefore trade information against speed. Video-based WAMs sit at

one end [50, 76, 85]: they forecast whole future frames or latents at uniform fidelity, pay for it in latency, and spend most of their capacity on appearance, while the quantity that decides contact, the target's metric pose when the action executes, remains implicit. Methods that forecast only dynamic regions or salient tokens [68, 95] reduce where they predict but still predict in image space, where that pose remains implicit as well. Target-centric predictors sit at the other end [19, 28, 51]: they are fast but blind to surroundings that can block or redirect the motion. What none of them provides is knowledge of the moving scene that is precise where contact depends on it, coarse elsewhere, and cheap to use at execution time.

Our key observation is that a dynamic policy needs to know very little precisely. Contact depends on the target's pose at the moment the action executes, to within millimeters, whereas the rest of the scene matters only through the clearance it leaves. Nor does the forecast have to be computed at test time: a forecast that supervises training shapes what the policy represents and can be dropped at deployment. Following this observation, we present SE(3)-WAM, a role-aware world–action model for dynamic manipulation (Fig. 2). Its input keeps only what motion requires: the current image, encoded by a frozen encoder, and short histories of the target's metric SE(3) pose and of the robot state, which convey motion directly instead of a stack of past frames. A world stream condenses these inputs into a small set of summary tokens and is trained to forecast, in one metric 4D space, the target's future SE(3) trajectory precisely and the rest of the scene as a sparse 3D flow. An action stream reads the summary tokens at every layer through shared attention, generates action chunks by conditional flow matching, and replans after every executed step, so its actions follow the target as it moves. The forecasts act only as supervision: they shape the summary tokens during training, are never fed back to the actions, and their heads are skipped at deployment. SE(3)-WAM thus gains the precision of a metric forecast at a latency close to that of a reactive policy.

We evaluate SE(3)-WAM on Dynamic-14, a simulated benchmark of fourteen tasks in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes. SE(3)-WAM achieves state-of-the-art success of [XX.X]%, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, and succeeds in [XX]% of trials across five real-robot tasks with linear, circular, and accelerating target motion, against [XX]% for the strongest baseline. Ablations show that the SE(3) pose history outperforms visual histories, that role-aware forecasts outperform target-centric and image-space forecasts, and that using the forecast only as a training signal matches feeding it to the actions while keeping deployment fast.

Our contributions are threefold:

- We identify that in dynamic manipulation, inference latency turns directly into contact error, and propose role-aware 4D modeling, which represents the target precisely by its metric SE(3) pose history and future trajectory and the rest of the scene coarsely by a sparse 3D flow.
- We present SE(3)-WAM, a world–action model whose action stream reads a representation shaped by these role-aware forecasts, while the forecasts themselves are never fed back and are skipped at deployment, obtaining the precision of prediction at a latency close to that of a reactive policy.
- We build Dynamic-14, a benchmark in which perception, scene motion, and execution share one physical clock. SE(3)-WAM achieves state-of-the-art results on it and on five real-robot tasks, and ablations confirm the contribution of each component.

2. Related Work

Dynamic manipulation. Acting on a moving target requires compensating for the time between sensing and acting, a problem as old as feedback control with dead time [65] and predictive tracking [4]. Classical catching and grasping systems estimate the target's trajectory with Kalman-type filters [5], servo on visual features [32], and plan an interception [2, 6, 27], later combined with grasp and reachability planning [1, 53]. These pipelines are precise but engineered for each object and task. Learned visuomotor policies [16, 99] and vision–language–action models (VLAs) [9, 11, 37, 56, 57, 107] generalize across objects and instructions, but they are trained and evaluated in quasi-static scenes, where the world waits while the model computes. Recent work brings learned policies to moving scenes. DOMINO provides a dynamic benchmark together with PUMA, which reads historical optical flow and is supervised with future object features [19]. DynamicManip synthesizes dynamic demonstrations from a single static one [41], DynamicWAM conditions a WAM on flow and kinematic descriptors of the past [50], and DynaWM refines a base VLA's action chunk for moving objects [14]. Other methods forecast salient visual tokens [68], a future image [96], a compact future code with the time to the next gripper event [51], or a 3D Gaussian velocity field that is optimized online [91], and feed the forecast to the policy at test time. Execution schemes reduce or hide the latency instead. DynamicVLA and StreamingVLA overlap inference with execution [61, 82], real-time chunking and bidirectional decoding keep consecutive chunks consistent [10, 47], asynchronous inference decouples prediction from control [63], VLASH rolls the robot state forward to the execution time [69], and Chronos Policy encodes observation and execution timestamps [73]. These schemes change when actions are computed and executed, whereas SE(3)-WAM changes what the policy represents, so the two are orthogonal. Closest to

ours, PUMA also uses future prediction only as a training signal. It forecasts mask-pooled image features of the target from 2D flow, whereas SE(3)-WAM forecasts the target's metric SE(3) trajectory and a sparse 3D flow of its surroundings.

World models and world–action models. World models learn how observations evolve, and they have been used for planning with visual foresight [21], for reinforcement learning in a learned latent space [24, 25], and for planning on pretrained visual features [3, 103]. In manipulation, predicted frames or subgoal images have been converted into actions [8, 17, 97], and video prediction has served as pretraining [80], as a predictive objective for inverse dynamics [71], and as a source of predictive features [29]. World–action models (WAMs) generate future video and actions jointly in one network [40, 76, 85, 104]. Recent studies compare how the two streams should exchange information [76, 87] and how long and how compressed their context should be [31, 48]. Because video is expensive and dominated by appearance, a growing body of work changes what is forecast: dynamic regions, depth, and semantic features [95], flow, geometry, and semantics alongside RGB [89], motion-weighted video [33], object masks [86] or object slots [49], optical flow [15] or motion vectors [42], compact future latents [38, 39], and targets in the policy's own condition or token space [67, 98]. A second question is whether the forecast is needed at test time. Fast-WAM finds that most of the benefit of video modeling remains when test-time imagination is skipped [90], and other methods drop their predictors [19, 67, 98] or auxiliary structure branches [89] after training. SE(3)-WAM follows this training-only use of the forecast but changes its target. Instead of pixels or latents at a uniform resolution, it forecasts a metric 4D state whose precision follows each entity's role: a full SE(3) trajectory for the target and a sparse 3D flow for the rest of the scene. Its action stream reads observation-derived summary tokens shaped by these forecasts, never the forecasts themselves, so the forecast heads are skipped at deployment.

Structured 3D and 4D representations. Metric 3D structure makes manipulation policies more precise and sample-efficient. Voxel, multi-view, and point-cloud policies act on scene geometry [22, 35, 62, 92], object-centric point clouds improve generalization [106], and 3D and 4D VLAs bring 3D and spatiotemporal inputs to large models [94, 100]. Motion has been exposed to policies through 2D point tracks [7, 78, 83, 101], dense pixel motion [55, 58, 59], and 3D flow [23, 88, 102], either as inputs or as intermediate predictions that are then turned into actions. Object-level representations include keypoints [52], object SE(3) trajectories [28, 66, 79], and separate encodings of the robot, the task object, and obstacles [18]. Rigid-body structure has long been used to learn scene dynamics, from SE(3) decompositions of point clouds [12, 13] and 3D dynamic scene memories [84] to scene flow estimation [46, 70], and PointWorld scales action-conditioned 3D point flow to in-the-wild robot data [30]. Point trackers, pose estimators, and feed-forward reconstruction models [34, 74, 77, 81, 93] now make such labels cheap to extract from video. These representations usually apply one granularity to the whole scene, either dense geometry and flow everywhere or poses for every object. SE(3)-WAM assigns granularity by role. The target, where contact depends on millimeters, is described by its SE(3) pose history and forecast, and the rest of the scene, which matters through the clearance it leaves, by a sparse 3D flow. For real robots, we pretrain on DROID with the 3D point tracks that PointWorld extracted from it [30].

3. SE(3)-WAM

SE(3)-WAM (Fig. 2) is a transformer with a world stream and an action stream that share attention. The world stream encodes the current observation and the recent motion and is trained to forecast a role-aware 4D scene. The action stream generates action chunks by conditional flow matching [43] and reads the world only through a few summary tokens. We first state the setting that motivates this design.

3.1. Acting on a Moving Target

Execution times. At query t, a policy receives an image, a depth map, and the robot state captured at time τ_t, together with histories of earlier states, and outputs a chunk $A_t = (\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ whose j-th command executes at

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + \ell_t + j\Delta_a, \tag{1}$$

where ℓ_t is the latency from capture to the first execution (preprocessing, inference, and actuation) and Δ_a is the control period. A command must fit the scene at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t (Fig. 3), and the lead $\Delta_{t,j} = \tau^{\mathrm{a}}_{t,j} - \tau_t$ grows with every millisecond of inference.

Target and context. Let $W_t(\mathbf{x}, \tau) \in \mathbb{R}^3$ be the displacement between τ_t and τ of the scene point at $\mathbf{x}$, in the robot base frame. The *target* $\mathcal{O}$ is the rigid body whose pose error enters a contact or containment tolerance ε, e.g., an object to grasp or a receptacle to place into. The *context* $\mathcal{C}$ contains the bodies that only need to be avoided, such as a belt or an occluder, with clearance m. On the target, the motion field is set by its pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$:

$$W_t(\mathbf{x}, \tau) = T(\tau)\, T(\tau_t)^{-1}\mathbf{x} - \mathbf{x}, \qquad \mathbf{x} \in \mathcal{O}. \tag{2}$$

Where the error comes from. Let a policy act, explicitly or implicitly, on an estimate $\hat{T}$ of the target pose at τ^{a}. For a contact point at lever arm r and a rotation error θ, the contact

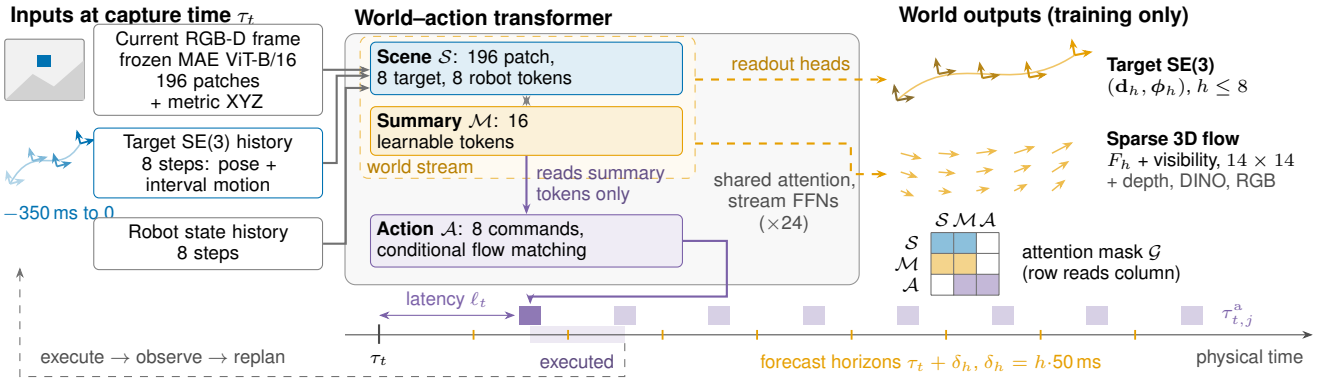


Figure 2. **SE(3)-WAM.** The world stream holds 212 scene tokens, the current image patches from a frozen encoder together with short histories of the target's metric SE(3) pose and of the robot state, and 16 summary tokens. During training, light heads read the world stream out into a role-aware 4D forecast at physical horizons $\tau_t + \delta_h$: the target's SE(3) trajectory and a sparse 3D flow with visibility on a 14×14 grid, plus appearance targets (dashed: training only). The action stream generates a chunk of 8 commands by flow matching. The attention mask $\mathcal{G}$ lets world tokens read only each other and action tokens read the summary tokens and each other, so the forecasts never reach the actions. The policy executes one command, observes, and replans.

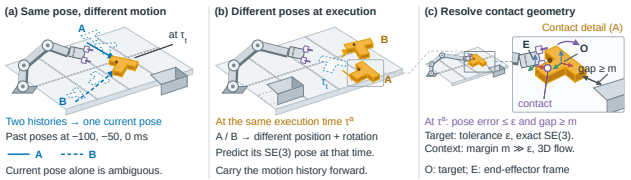


Figure 3. **Acting on a moving target** (schematic). (a) Two histories reach the same pose at capture time τ_t but imply different motions. (b) At the execution time τ^{a}, they lead to different target poses. (c) A contact requires the end-effector frame E to match the target O within ε, whereas the context matters only through the gap it leaves, $m \gg \varepsilon$.

error is at most $\|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| + 2r\sin(\theta/2)$. Writing the motion over the lead Δ as constant velocity $\mathbf{v}$ plus a residual $\boldsymbol{\rho}$ (acceleration, rolling, impacts) gives

$$\|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| \le e_{\mathrm{loc}} + \Delta\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \quad (3)$$

the errors of the current position, of the velocity times the lead, of the timing times the speed, and of the residual motion (Appendix G). For the context, a collision requires a prediction error of at least m somewhere along the commanded path. Two conclusions follow. First, the precision that matters is concentrated on the target: a millimeter-level estimate of its pose and motion, including rotation through the lever arm r, whereas the context can be predicted far more coarsely. Second, latency enters the budget twice, through the lead Δ that multiplies every velocity error and through the motion that the residual accumulates, so information that costs inference time pays for itself only if it removes more error than the extra latency adds. These conclusions shape the three parts of SE(3)-WAM: what it remembers (Sec. 3.2), what it learns to forecast (Sec. 3.3), and how its actions use this forecast without paying for it at deployment (Sec. 3.4).

3.2. Inputs: The Current Frame and Motion Histories

SE(3)-WAM sees only one image. A frozen MAE ViT-B/16 [26] encodes the current RGB frame into 196 patch tokens, and the depth map gives each patch the metric 3D position of its center point in the base frame. The motion comes from two short histories over the last 8 control steps (350 ms at 20 Hz). The target history is the current target pose in the base frame together with the seven interval motions between consecutive samples: the translation, the relative rotation, the time interval, and the finite-difference velocity and acceleration, in physical units (Appendix A.1). It forms 8 target-state tokens. The robot history forms 8 end-effector state tokens. Together with the patch tokens, these are the 212 scene tokens. On the real robot, a causal tracker estimates the target pose in every frame: SAM 2 propagates a mask from a prompt on the target [60], and FoundationPose tracks the object from a model or a few reference views [77]. Each estimate is emitted once with its capture time, and later frames do not revise it (Appendix A.2). In simulation, we read the target pose from the simulator, which separates the policy from tracking errors, and we evaluate pose noise and tracked poses separately (Appendix F.2). The model receives no past images, so its input cost does not grow with the history.

3.3. Role-Aware 4D Forecast

The world stream is trained to forecast W_t at horizons $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50$ ms and $h = 1, \ldots, 8$, which span the execution times of a whole chunk (400 ms). We parameterize

the target through Eq. (2). For each horizon,

$$\mathbf{d}_h = \mathbf{p}(\tau_t+\delta_h) - \mathbf{p}(\tau_t), \quad \boldsymbol{\phi}_h = \mathrm{Log}\big(R(\tau_t+\delta_h)R(\tau_t)^\top\big)^\vee, \tag{4}$$

and a pose head regresses $(\mathbf{d}_h, \boldsymbol{\phi}_h)$ with a squared translation error plus λ_R times the squared geodesic rotation error ($\mathcal{L}_P$). The rest of the scene is a sparse 3D flow: for the center point of each of the 196 patches on the 14×14 grid, about [X] mm apart in the workspace, a flow head regresses its base-frame displacements $F_h \in \mathbb{R}^3$ and its visibility at every horizon ($\mathcal{L}_F$). The grid is deliberately coarse, since the context is tolerated up to its clearance, and the target's precision comes from its pose rather than from its points. As further training targets, appearance heads regress the future depth, the features of a frozen DINOv3 teacher [64], and the RGB of moving surfaces, and a reconstruction head restores the target region of the current image ($\mathcal{L}_{\text{aux}}$). All heads are light readouts of the final hidden states of the world stream, all labels are cumulative from the current state, and the losses use valid labels only.

3.4. Actions and the Shared Backbone

The world stream holds the 212 scene tokens and 16 learnable summary tokens. The action stream holds 8 tokens, one per command, each embedding a noisy command and the flow time. A chunk spans 8 control periods of $\Delta_a = 50$ ms, and each command is an end-effector translation and rotation plus a gripper command. The attention mask $\mathcal{G}$ lets world tokens attend only to world tokens and lets action tokens attend to the summary tokens and to each other. The actions therefore read the world only through the summary tokens, whose content the forecast losses shape during training, while the forecasts themselves never enter the action stream. Both streams share the attention projections of each layer and have separate feed-forward networks, so they exchange information in attention but keep stream-specific capacity. For a normalized chunk $\mathbf{a}_1$, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and $u \sim \mathcal{U}[0, 1]$, the action loss is

$$\mathcal{L}_A = \mathbb{E}\left\|v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0)\right\|^2, \tag{5}$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$ and the scene tokens $\mathcal{S}_t$ of query t. The total loss is $\mathcal{L} = \mathcal{L}_A + \lambda_F\mathcal{L}_F + \lambda_P\mathcal{L}_P + \lambda_{\text{aux}}\mathcal{L}_{\text{aux}}$. Sec. 4.4 compares this coupling with alternatives, which we label as follows: R0 disables the forecast losses; R2 adds world queries whose hidden states the actions read at every layer; R3 lets these queries read the actions, and R4 opens both directions; and R5 encodes the decoded forecast as geometric tokens for the actions (Appendix A.5).

Deployment. At test time, the forecast heads are skipped. Because world tokens never attend to actions, the world stream runs once per query, and the action stream integrates 5 Euler steps on its cached keys and values. The robot executes the first command, the policy observes again, and it replans, so every executed command is generated from an observation taken only ℓ_t earlier. The p95 time-to-action is [XX] ms on an RTX 5090.

Implementation. The transformer has 24 layers of width 1024 with 16 heads and feed-forward width 4096, with 504.0M backbone parameters and 523.8M trainable parameters in total. The MAE ViT-B/16 encoder (85.8M) and the DINOv3 teacher (303M) stay frozen. On Dynamic-14, we train on all windows of 800 demonstrations per task for 20 epochs (51,740 updates at a global batch size of 512) with AdamW and a cosine schedule, with $\lambda_F = \lambda_P = 1$, $\lambda_R = \lambda_{\text{aux}} = 0.1$, which takes [X] hours per training seed on [N] RTX PRO 6000 GPUs (Appendix A.6).

3.5. Pretraining for Real Robots

Real robot data do not provide object poses, so the real-robot base learns the parts of the forecast that DROID [36] supports. It has the same architecture with 48 layers (1.02B trainable parameters) and the same inputs except the target history. Its targets are the actions, the sparse 3D flow, and the future DINOv3 features. The flow labels come from the 3D point tracks and the optimized camera calibrations that PointWorld released for DROID [30], resampled to one track per image patch and to DROID's native timing. We train on 598,889 windows from 42,087 episodes for 21,060 updates on 64 GPUs (Appendix B). Each real-robot policy initializes from this base, adds the target-state tokens and the pose head, and is fine-tuned on its task with tracked poses. The simulation model is trained from the generic initialization, a frozen MAE encoder with a randomly initialized transformer.

4. Experiments

We compare SE(3)-WAM with reactive, dynamic, and world–action policies in simulation and on a real robot (Secs. 4.2 and 4.3) and then ablate its history, its forecast, and the way its actions use the forecast (Sec. 4.4).

4.1. Setup

Dynamic-14. We built fourteen tasks in MuJoCo/robosuite [72, 105] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Appendix C, Fig. S3). Sweeps vary the speed (0.5–1.5×), the added latency (0–200 ms), and the passage clearance ([values] mm). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report every Dynamic-14 number on 1,400 held-out scenes (100 per task, new seeds). Our variants average three training seeds, and

each baseline is trained once. We give paired 95% intervals from a bootstrap over held-out scenes.

Protocol. Observation, scene motion, and execution share one physical clock with a control period of 50 ms. All methods run under the same executor, and physics advances while a method computes, by its measured latency on one RTX 5090. A policy is queried again once a command has executed and its previous inference has finished, and until a new chunk arrives, the executor continues with the previous one. The +150 ms setting adds 150 ms of delay for every method, and the paused setting stops physics during inference (Appendix C.3). We report the task-balanced success rate (SR), the contact-time error e_t, the position error at first contact e_p, and the p95 time-to-action.

Baselines. We compare with reactive policies (ACT [99], Diffusion Policy [16], and $\pi_{0.5}$ [57]), dynamic VLAs (DynamicVLA [82] and PUMA [19]), image- or latent-space WAMs (DreamVLA [95] and OpenWAM-α [76]), and a dynamic WAM (DynamicWAM [50]). Each baseline uses its official code, inputs, and recipe, is trained on the same demonstrations, and receives a rendered wrist camera if its recipe uses one (Appendix D). SE(3)-WAM uses only the agent view, but it also uses depth and the target pose. To separate these inputs from the model, $\pi_{0.5}$ and DynamicWAM also receive our SE(3) history tokens (+SE(3)), and two baselines use the pose history alone: *CV input* is SE(3)-WAM without forecast losses that receives the constant-velocity extrapolation of the target pose as input tokens, and a Kalman interceptor plans to the target pose predicted by a constant-acceleration Kalman filter [5].

4.2. Main Results

SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 1), [X.X] points above the strongest baseline, [name of the strongest baseline], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name]. The margin is largest on [rotating targets and on fast conveyor and translating targets], where the target moves farthest during the lead. Its p95 time-to-action of [XX] ms is [X]× lower than that of the image-space WAMs, whose latency adds to the lead they must bridge. With 150 ms of added delay, SE(3)-WAM keeps [XX.X]%, whereas reactive policies drop to [XX.X]% and image-space WAMs to [XX.X]%. With physics paused during inference, the difference to [name of the strongest paused baseline] is [±X.X] points, which separates the contribution of speed from that of the representation. The CV-input variant and the Kalman interceptor trail SE(3)-WAM by [X.X] and [X.X] points, mostly on non-constant motion, so the learned model does more than extrapolate the pose. Results on DOMINO [19] and on static LIBERO tasks [44] are in Appendix D.2.

Figure placeholder: Fig. 4 — real-robot trials

Rows: a success of SE(3)-WAM on the accelerating task; the strongest baseline on the same trial key; a failure of SE(3)-WAM. **Columns:** the query frame, the first execution time τ^{a}, and contact. **Overlays:** blue tracked past poses; purple commanded gripper path. Real photos, unretouched; overlays from logged data of each trial.

Figure 4. **Real-robot trials.** SE(3)-WAM closes where the target will be when the command executes, whereas the baseline closes behind it. Bottom: a failure after the tracker lost the target under the gripper.

4.3. Real-Robot Experiments

We evaluate five tasks on a seven-DoF Franka arm with a parallel-jaw gripper and a calibrated RGB-D camera (Fig. S4). The target moves along a line in two tasks ([T1, T2]), on a circle in two tasks ([T3, T4]), and with acceleration in one task ([T5]) (Table 2, Appendix D.3). Each method is fine-tuned per task on the same [N] teleoperated demonstrations and runs [20] trials per task with shared trial keys. SE(3)-WAM tracks the target with FoundationPose [77] and runs on one RTX 5090 with a p95 time-to-action of [XX] ms. It succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline] and [XX.X]% without the DROID base (Table 2). Its advantage is largest on [accelerating motion], where extrapolating the last observed velocity fails, and its remaining failures are mostly [late closures after tracking losses under the gripper] (Fig. 4).

Pretraining. The DROID base raises the real-robot SR by [X.X] points over the generic initialization (Table 2), and a base pretrained with actions only recovers [XX]% of this gain at the same budget, so most of the benefit comes from the 3D flow and DINO targets (Appendix B).

4.4. Ablations: What Does a Dynamic Policy Need?

Each variant changes one component of the final model and keeps the data, the training budget, and the other components fixed (Table 3 and Fig. 5). The complete comparisons are in Appendix F.

What to remember. We keep the current frame, the current target pose, and the robot history fixed. Adding eight frames of RGB, optical flow over the same window with DynamicWAM's motion statistics [50], or video-VAE latents changes SR by [X.X] to [X.X] points and raises the time-to-action by up to [X]×, whereas the SE(3) history adds [X.X] points at almost no cost (Table 3). The gain appears once the lead exceeds the critical lead $\varepsilon/\|\mathbf{v}\|$ of a family, grows with

Table 1. **Main results.** Held-out SR (%) on Dynamic-14 (1,400 new scenes, task-balanced, real time, three training seeds for our variants and one for baselines) by family, with 150 ms of added delay, and with physics paused during inference, together with the contact-time error e_t (ms), the position error at first contact e_p (mm), and the p95 time-to-action on one RTX 5090. Inputs: R, RGB and robot state. D, depth. P, target SE(3) history. Best in bold. Intervals in Appendix E.

		Dynamic-14 SR, real time unless noted										
Method	Inputs	All	Grasp	Place	Pass	Capture	Impact	+150 ms	Paused	e_t	e_p	p95 (ms)
ACT [99]	R	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy [16]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ [57]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA [82]	R	—	—	—	—	—	—	—	—	—	—	—
PUMA [19]	R	—	—	—	—	—	—	—	—	—	—	—
DreamVLA [95]	R	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α [76]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM [50]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	RD+P	—	—	—	—	—	—	—	—	—	—	—
Kalman interceptor	D+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	RD+P	—	—	—	—	—	—	—	—	—	—	—

Figure placeholder: Fig. 5 — ablations (four panels)

(a) History. x: added latency (0–200 ms); y: ΔSR vs. current frame only. Lines: RGB window, optical flow, video-VAE latent (gray), SE(3) history (blue), SE(3) history with pose noise (blue dotted); stationary targets as a faint line; ticks at the families' critical lead $\varepsilon/\|\mathbf{v}\|$. **(b) Target forecast.** Bars by motion type (constant velocity, harmonic, rolling/sliding, post-impact): forecast error at τ^{a} of CV, CA, and the learned SE(3) forecast (orange), with SR of the corresponding policies as dots. **(c) Context.** Passage tasks only. x: clearance (mm); y: SR; lines: no context flow, 14^2, 28^2, 56^2 grids, future latent; markers: each grid's measured path error. **(d) Coupling.** x: p95 time-to-action (ms); y: SR at 0 and 150 ms added latency; points: R0, SE(3)-WAM, R2, R4, R5. 95% paired bootstrap bands, held-out scenes, three seeds.

Figure 5. **Ablations.** (a) The SE(3) history helps once the lead exceeds the critical lead, and more with latency, while visual histories stay flat. (b) The learned SE(3) forecast beats extrapolation on non-constant motion. (c) A coarse context flow suffices until the clearance reaches its path error. (d) Reading or decoding the forecast at test time adds latency but not success.

Table 2. **Real-robot results.** Success rate (%) on five tasks, grouped by the target's motion, [20] trials per task and method, and over all trials. Every method is fine-tuned per task on the same demonstrations. Binomial 95% intervals are given in Appendix D.3.

	Linear		Circular		Accel.	
Method	[T1]	[T2]	[T3]	[T4]	[T5]	All
$\pi_{0.5}$ [57]	—	—	—	—	—	—
DynamicWAM [50]	—	—	—	—	—	—
SE(3)-WAM, no DROID base	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

latency, and vanishes for stationary targets (Fig. 5a). Frozen readouts show that visual histories do contain the motion (Appendix F.2), but at our data scale the policy does not turn it into metric velocity. With pose noise of a real tracker, the SE(3) history keeps [XX]% of its gain. The same history given to $\pi_{0.5}$ and DynamicWAM raises their SR by [X.X] and [X.X] points (Table 1), so the benefit is not specific to our model.

What to predict. Disabling all forecast losses (R0) costs [X.X] points, and the gap grows with the lead. Forecasting only the target's SE(3) trajectory costs [X.X] points, mostly on passage and placement tasks, where the context constrains the motion, and forecasting only the context flow costs [X.X], mostly on rotating targets, where the lever arm amplifies rotation errors. Replacing the target's pose by the flow of its points, at the same uniform granularity as the context, costs [X.X] points. At the execution times, the learned SE(3) forecast is as accurate as constant-velocity extrapolation on conveyor motion but [X] times more accurate on rolling,

Table 3. **Ablations** on held-out Dynamic-14 scenes (three training seeds). Each block changes one component of SE(3)-WAM (⋆). SR, SR_{150}: success (%) without and with 150 ms of added delay. e_t: contact-time error (ms). e_p: position error at first contact (mm). p95: time-to-action (ms).

Variant	SR	SR_{150}	e_t	e_p	p95
SE(3)-WAM ⋆	—	—	—	—	—
History (current frame and pose kept)					
none	—	—	—	—	—
RGB window (8 frames)	—	—	—	—	—
optical flow, motion statistics	—	—	—	—	—
video-VAE latent	—	—	—	—	—
SE(3) with tracker noise	—	—	—	—	—
Forecast					
none (forecast losses off, R0)	—	—	—	—	—
target SE(3) only	—	—	—	—	—
context flow only	—	—	—	—	—
uniform: point flow on target	—	—	—	—	—
image space: future RGB/latent	—	—	—	—	—
no appearance targets	—	—	—	—	—
Use of the forecast at test time					
action reads world queries (R2)	—	—	—	—	—
world reads actions (R3)	—	—	—	—	—
bidirectional (R4)	—	—	—	—	—
action from decoded forecast (R5)	—	—	—	—	—

Figure placeholder: Fig. 6 — forecasts at the execution time

Rows: a rolling capture (median forecast error) and a post-impact grasp that fails. **Columns:** the query frame at τ_t, the frame at τ^a, and a 3D inset in the base frame. **Overlays:** blue past poses, orange forecast triads and coarse flow arrows from the training-time heads, gray ground truth at the same times, purple commanded path. Full set in Fig. S5.

Figure 6. **Forecasts at the execution time** on held-out Dynamic-14 episodes, read from the training-time heads. Top: the SE(3) forecast follows a rolling target that extrapolation overshoots. Bottom: before an impact, the regressed forecast averages the possible outcomes, and the grasp misses.

sliding, and post-impact motion (Fig. 5b). For the context, the 14×14 flow, with a path error of [X] mm, matches 28^2 and 56^2 grids until the passage clearance falls to [X] mm (Fig. 5c). Image-space futures, i.e., future RGB or video latents in place of the 4D forecast, reach [XX.X]% at [X]× the training cost. As auxiliary targets next to the 4D forecast, however, depth, DINO features, and moving-region RGB add [X.X] points, and like the 4D heads they are skipped at deployment.

How to use the forecast. Letting the actions read the hidden states of world queries (R2), letting the world also read the actions (R3, R4), or generating the actions from the decoded forecast (R5) changes SR by [±X.X] to [±X.X] points, within the seed variation, at [X.X]–[X.X]× the time-to-action of SE(3)-WAM (Fig. 5d). Under 150 ms of added delay, the differences stay within [±X.X] points. The forecast thus helps as a training signal that shapes what the actions read, as Fast-WAM found for static scenes [90], and generating it at test time adds latency without adding success. Separate parameters for the world and action streams instead of shared attention change SR by [±X.X] points at matched size (Appendix F.1).

5. Conclusion

We presented SE(3)-WAM, a world–action model for manipulating moving objects that spends precision where contact depends on it and time only where it pays off. It remembers the target's metric SE(3) motion instead of past frames, learns to forecast the target's SE(3) trajectory precisely and the rest of the scene as a sparse 3D flow, and lets its actions read a representation shaped by these forecasts without generating them at test time. On Dynamic-14, where the scene keeps moving while a policy computes, and on five real-robot tasks, it improves on reactive, dynamic, and world–action policies. Our ablations attribute the gains to the metric motion history and to role-aware forecasts used as a training signal, rather than to richer appearance models or to reading the forecast online. For world–action models in dynamic settings, they favor forecasting the few quantities that decide contact and keeping the deployed policy as fast as a reactive one.

Limitations. SE(3)-WAM assumes a rigid target that moves on its own until contact, so deformable or articulated targets and post-contact dynamics are outside its scope. It depends on a pose estimate of the target, and tracker failures propagate to the policy; in simulation we read the pose from the simulator. The regressed target forecast cannot represent multimodal futures, e.g., before an impact. Our real-robot base learns from DROID without target poses, so its pose-related parts are learned only during fine-tuning. Finally, our tasks have a single target and slow context motion.

LLM Usage Statement

[To be completed before submission: actual tools used, affected parts of the research and manuscript, and extent of use.]

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 3

[4] Ronald Azuma and Gary Bishop. Improving static and dynamic registration in an optical see-through HMD. In *Proceedings of SIGGRAPH*, pages 197–204, 1994. 2

[5] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. 2, 6, 23, 35

[6] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[7] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[8] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, 2024. 3

[9] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1, 2

[10] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, 2025. 2

[11] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, 2023. 2

[12] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, 2017. 3

[13] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[14] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[15] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, Kai Wang, Jianhua Yang, Dong An, He Guan, Gaoteng Liu, Jianlou Si, Jun Huang, Jing Liu, Nianfeng Liu, Yan Huang, and Liang Wang. FlowWAM: Optical flow as a unified action representation for world action models. *arXiv preprint arXiv:2607.13017*, 2026. 3

[16] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 2, 6, 7, 24

[17] Yilun Du et al. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, 2023. 3

[18] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. *arXiv preprint arXiv:2503.11565*, 2025. 3

[19] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 1, 2, 3, 6, 7, 22, 23, 24

[20] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 29

[21] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation*, 2017. 3

[22] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. RVT: Robotic view transformer for 3D object manipulation. In *Conference on Robot Learning*, pages 694–710, 2023. 3

[23] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *arXiv preprint arXiv:2505.10075*, 2025. 3

[24] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059), 2025. 3

[25] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 3

[26] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 4, 14, 28

[27] B. Hove and Jean-Jacques E. Slotine. Experiments in robotic catching. In *American Control Conference*, 1991. 2

[28] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2, 3

[29] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025. 3

[30] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3, 5, 16

[31] Wei Huang, Bohan Zhang, Chenzhi Liu, Isabella Liu, Shuai Yang, Weian Mao, Luozhou Wang, Yicheng Xiao, Weifeng Lin, Qixin Hu, Bryan Chu, Sifei Liu, Linxi Fan, Xiaojuan Qi, Song Han, and Yukang Chen. Long-WAM: Scaling the context of world-action models. *arXiv preprint arXiv:2610.10528*, 2026. 3

[32] Seth Hutchinson, Gregory D. Hager, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996. 2

[33] Haoyuan Ji, Lingxiang Fan, Shang Su, Yinqiao Lu, Mengkai Shi, Jun Gao, and Shuo Feng. DC-WAM: Dynamic-centric visual supervision and reasoning for world-action models. *arXiv preprint arXiv:2607.25918*, 2026. 3

[34] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together. In *European Conference on Computer Vision*, 2024. 3

[35] Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3D diffuser actor: Policy diffusion with 3D scene representations. In *Conference on Robot Learning*, 2024. 3

[36] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 5, 16

[37] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024. 1, 2

[38] Fenghao Lei, Zhixiong Huang, Long Yang, Jiabao Chen, Peilin Huang, Han Fu, Zhuo Li, and Xiaoxue Ren. DELEw0.5: Inferring action from future latent state for robotic manipulation. *arXiv preprint arXiv:2608.22067*, 2026. 3

[39] Jiajun Li, Tiecheng Guo, Yifan Ye, Rongyu Zhang, Xiaowei Chi, Qianpu Sun, Ying Li, Yunfan Lou, Jinqiao Wang, Zhihe Lu, Meng Guo, and Shanghang Zhang. Efficient-WAM: A 1B-parameter world-action model with low-cost future imagination. *arXiv preprint arXiv:2606.10040*, 2026. 3

[40] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 3

[41] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[42] Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3

[43] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 3

[44] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023. 6, 23

[45] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 14

[46] Xingyu Liu, Charles R. Qi, and Leonidas J. Guibas. FlowNet3D: Learning scene flow in 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019. 3

[47] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, 2025. 2

[48] Yushan Liu, Jingjing Fan, Shoujie Li, Yifan Xie, Xiao-Ping Zhang, and Wenbo Ding. PACT-WAM: Predicting actions and visual foresight with compact temporal encoding for robot manipulation. *arXiv preprint arXiv:2602.15882*, 2026. 3, 29

[49] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 3

[50] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 7, 24, 29

[51] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[52] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *International Symposium on Robotics Research*, 2019. 3

[53] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[54] Henrique Morimitsu, Xiaobin Zhu, Roberto M. Cesar, Xiangyang Ji, and Xu-Cheng Yin. DPFlow: Adaptive optical flow estimation with a dual-pyramid framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17810–17820, 2025. 17

[55] E-Ro Nguyen, Yichi Zhang, Kanchana Ranasinghe, Xiang Li, and Michael S. Ryoo. Pixel motion diffusion is what we need for robot control. *arXiv preprint arXiv:2509.22652*, 2025. 3

[56] Octo Model Team, Dibya Ghosh, Homer Walke, et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[57] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 6, 7, 24

[58] Kanchana Ranasinghe, Xiang Li, E-Ro Nguyen, Cristina Mata, Jongwoo Park, and Michael S. Ryoo. Pixel motion as universal representation for robot control. *arXiv preprint arXiv:2505.07817*, 2025. 3

[59] Kanchana Ranasinghe, Honglu Zhou, Yu Fang, Luyu Yang, Le Xue, Ran Xu, Caiming Xiong, Silvio Savarese, Michael S. Ryoo, and Juan Carlos Niebles. Future optical flow prediction improves robot control & video generation. *arXiv preprint arXiv:2601.10781*, 2026. 3

[60] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 4, 14

[61] Yiran Shi, Dongqi Guo, Tianchen Zhao, Feng Gao, Liangzhi Shi, Chao Yu, ZhiJian Mo, Qihua Xiao, XiaoShuai Peng, Qingmin Liao, and Yu Wang. StreamingVLA: Streaming vision-language-action model with action flow matching and adaptive early observation. *arXiv preprint arXiv:2603.28565*, 2026. 2

[62] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, 2022. 3

[63] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[64] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 5, 14, 15, 28, 31

[65] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[66] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. *arXiv preprint arXiv:2601.05237*, 2026. 3

[67] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026. 3

[68] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. 1, 2

[69] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[70] Zachary Teed and Jia Deng. RAFT-3D: Scene flow using rigid-motion embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. 3

[71] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, 2025. 3

[72] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 18

[73] Changhao Wang, Jaden Clark, and Shuran Song. Chronos policy: Learning robot policies on physical time. *arXiv preprint arXiv:2610.13929*, 2026. 2

[74] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 3

[75] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025. 17

[76] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 3, 6, 7, 24

[77] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 3, 4, 6, 14

[78] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for

policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[79] Gaoyuan Wu, Ziyu Shan, Haoyang Du, Yuyao Jiang, and Ziwei Wang. AffordTrajDP: Dynamic affordance-guided visuomotor policy learning for robotic manipulation. *arXiv preprint arXiv:2608.01603*, 2026. 3

[80] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024. 3

[81] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. SpatialTracker: Tracking any 2D pixels in 3D space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 3

[82] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 2, 6, 7, 24

[83] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, 2024. 3

[84] Zhenjia Xu, Zhanpeng He, Jiajun Wu, and Shuran Song. Learning 3D dynamic scene representations for robot manipulation. In *Conference on Robot Learning*, 2020. 3

[85] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2, 3

[86] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 3

[87] Heng Yu, David D. Yuan, Juze Zhang, Changan Chen, Yao Feng, Michelle Baldonado, Steve Cousins, Li Fei-Fei, Jiajun Wu, and Ehsan Adeli. OpenWAM: An open framework for composable world-action models. *arXiv preprint arXiv:2610.07922*, 2026. 3

[88] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2024. 3

[89] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 3

[90] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 3, 8

[91] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. *arXiv preprint arXiv:2607.01938*, 2026. 2

[92] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[93] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one D4RT at a time. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3

[94] Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yu-Jie Yuan, Xinyue Cai, Guowei Huang, Xingyue Quan, Hang Xu, and Li Zhang. 4D-VLA: Spatiotemporal vision-language-action pretraining with cross-scene calibration. In *Advances in Neural Information Processing Systems*, 2025. 3

[95] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xin-Qiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 3, 6, 7, 24

[96] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[97] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 3

[98] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 3

[99] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 2, 6, 7, 24

[100] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning*, 2024. 3

[101] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 3

[102] Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 3DFlowAction:

Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. 3

[103] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, 2025. 3

[104] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 3

[105] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 18

[106] Yifeng Zhu, Zhenyu Jiang, Peter Stone, and Yuke Zhu. Learning generalizable manipulation policies with object-centric 3D representations. In *Conference on Robot Learning*, 2023. 3

[107] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023. 1, 2

Supplementary Material

This supplement gives implementation details of SE(3)-WAM (Appendix A), pretraining on real robot data (Appendix B), the benchmark (Appendix C), the baselines, external benchmarks, real robot, and timing (Appendix D), additional results (Appendix E), the complete ablation study behind Sec. 4.4 (Appendix F), and the error analysis that motivates the design (Appendix G). Colored dashes and bracketed entries denote values pending the final runs.

A. Implementation

A.1. Scene and Summary Tokens

Current image. A frozen MAE ViT-B/16 [26] encodes the 224×224 agent-view RGB into 196 patch tokens. We selected it over frozen DINOv3 encoders [64] and a CNN trained with the policy on development scenes (Appendix F.1). Each patch receives a view embedding, a linear embedding of its mean valid metric depth, and a linear embedding of the back-projected base-frame position of its center pixel, which we compute with the camera intrinsics and extrinsics. These positions also enter 3D rotary position embeddings for the patch queries and keys. The same center points are the source points of the sparse 3D flow (Appendix A.3). Target, robot, summary, and action tokens receive no rotary embedding. Past images never enter the model, and we drop their slots before the backbone, so the scene holds 212 tokens.

Target-motion history. At each query, the target history covers the last 8 control steps, the offsets $-350, -300, \ldots, 0$ ms. Each of the 8 target-state tokens is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$. Slots 0–6 describe the seven intervals between consecutive offsets, and slot 7 holds the current absolute pose in the base frame. Velocities divide interval displacements by the elapsed time, and accelerations difference adjacent velocities. We standardize each dimension with training statistics, zero the invalid dimensions, and append three validity flags (pose, interval, and acceleration) before a $28 \to 1024 \to 1024$ GELU MLP with a time embedding. When the pose source reports the target as lost, a learned embedding replaces the pose. In simulation, the poses come from the simulator. On the real robot, they come from the causal tracker of Appendix A.2, whose track we resample at the same offsets, interpolating linearly in translation and along the geodesic in rotation, without extrapolation and without interpolating across gaps longer than two camera periods.

Table S1. **Causal tracker accuracy.** Median and p95 error of the emitted poses against ground truth, fraction of frames reported as lost, and held-out Dynamic-14 SR of SE(3)-WAM with each pose source (*: main simulation setting; the real robot uses FoundationPose).

Tracker	mm	°	Lost	SR
FoundationPose, full res.	—	—	—	—
FoundationPose, 224×224	—	—	—	—
Simulator poses (oracle)*	0/0	0/0	0	—
Real robot, CAD prior	—	—	—	–
Real robot, reference views	—	—	—	–

Robot state. 8 end-effector states (position, rotation vector, and measured gripper opening) at the same offsets form the robot tokens.

Summary tokens. 16 learned embeddings form the summary tokens of the world stream. They attend to the scene tokens and to each other at every layer, and they are the only world tokens that the action tokens attend to.

A.2. Pose Tracker

On the real robot, SAM 2.1 (Hiera-small) propagates the target mask causally from a box in the first frame [60], which Grounding DINO gives for the object phrase of the instruction [45]. FoundationPose registers and tracks the object in the full-resolution RGB-D frame ([640 × 480]) with its released weights [77], using a CAD model or a set of [16] masked reference views captured once per object. The tracker runs in its own thread, and each estimate carries its capture time, its availability time, and a validity flag. The flag is false after a registration failure, a depth dropout over the mask, or a jump of more than [X] mm between frames. Each estimate is emitted once, and later frames do not revise it. Table S1 reports the tracker's accuracy on recorded episodes with motion-capture ground truth and on Dynamic-14 replays, which also calibrate the pose-noise model of Appendix F.2.

A.3. Forecast Heads

All forecast heads are light readouts of the final hidden states of the world stream. They are trained jointly with the action and skipped at deployment, and no token reads their outputs, so they shape the world stream only through the losses.

Target. An MLP on the pooled final states of the target-state and summary tokens reads out the trajectory $\{(\mathbf{d}_h, \phi_h)\}_{h=1}^{8}$. With valid horizons $\mathcal{V}$,

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \left[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \right], \quad \text{(S1)}$$

with $\lambda_R = 0.1$ and $\hat{R}_h = \mathrm{Exp}(\hat{\phi}_h^{\wedge})R(\tau_t)$. We use the left-relative rotation vector and the cumulative translation instead of a twist exponential, which would couple translation to rotation through its Jacobian. The forecast is a conditional mean, so it averages over multimodal futures (e.g., the outcome of an impact before it happens).

Sparse 3D flow. The flow is the cumulative 3D displacement of the $N = 196$ patch center points over the 8 horizons, $F \in \mathbb{R}^{N\times 8\times 3}$, in the base frame, together with a visibility $o \in \{0,1\}^{N\times 8}$. A linear readout on each patch token gives its point's displacement and visibility logit at every horizon. We divide the displacements by one fixed scale fitted on the training labels and use

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} o_{ih} \left\| \hat{F}_{ih} - F_{ih} \right\|^2 + \mathrm{BCE}(\hat{o}, o), \quad \text{(S2)}$$

where $\mathcal{V}_F$ holds the valid labels, so that an occluded point supervises only its visibility. The points cover the target, the robot, and the context alike; the target's precision comes from its pose head.

Appearance targets. Patch decoders on the final patch tokens predict three targets at every horizon: the metric depth of their patch (squared error on valid depth), the features of a frozen DINOv3 ViT-L/16 teacher [64] on the 14×14 grid after channel-wise layer normalization (squared error), and the RGB of moving surfaces (i.e., surfaces that move by more than 1 mm, including the robot), with a squared error inside that mask. A linear decoder on the current patch tokens reconstructs the RGB inside the target's current box, which ties the patch features to the target without adding an input. Each of these four targets enters $\mathcal{L}_{\mathrm{aux}}$ with weight 0.1.

A.4. Action Stream

Each chunk has 8 seven-dimensional commands, each with a base-frame translation increment, a left-relative rotation vector, and a gripper command, at a control period of 50 ms. Normalization uses the mean and the population standard deviation of valid, non-repeated issued commands in the training split. Flow matching uses a unit Gaussian in these coordinates with one u per chunk, and the action tokens receive u through adaptive layer normalization. Each training window draws 8 independent noise samples for its chunk and averages their losses, which reduces the variance of the action gradient without enlarging the batch. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip translation and rotation to 0.10 m and 0.5 rad per axis. The gripper follows the sign of its command.

A.5. Backbone and Coupling

Backbone. Each of the 24 layers applies one attention to all tokens with shared projections, followed by separate feed-forward networks (ratio 4) for the world tokens and the action tokens. World tokens use layer normalization and action tokens adaptive layer normalization, for 504.0M trainable backbone parameters. The alternative of Appendix F.1 gives the two streams separate attention projections, normalization, and feed-forward networks at a matched parameter count.

Attention mask. The mask acts on three token groups, the scene ($\mathcal{S}$), the summary tokens ($\mathcal{M}$), and the actions ($\mathcal{A}$). Scene and summary tokens attend to $\mathcal{S}$ and $\mathcal{M}$, and action tokens attend to $\mathcal{M}$ and $\mathcal{A}$. A blocked edge has the logit $-\infty$ at every layer and head, both in training and at deployment. Because the world tokens attend neither to the actions nor to the flow time, their states are computed once per query and reused across the Euler steps, and no future label reaches the action except through the gradients of the forecast losses on the shared parameters.

Ablated couplings. The variants of Sec. 4.4 use the same modules, initialization, and losses and change only the mask or add a small module. R0 disables every forecast loss. R1 adds one object query and one query per patch to the world stream, which attend to $\mathcal{S}$, $\mathcal{M}$, and each other and carry the forecast readouts instead of the pooled states, but opens no edge between them and the actions. R2 lets the action tokens attend to these queries at every layer. R3 lets these queries attend to the actions, and R4 opens both edges; in R3 and R4, the world states are recomputed at every Euler step. R5 decodes the forecast once per query and encodes the target trajectory and the flow with visibility probabilities, with rotations in a continuous 6D form, through a geometry encoder (0.37M parameters) into geometry tokens that the action attends to in addition to $\mathcal{M}$. R5 trains on the predicted geometry and backpropagates the action loss through it without teacher forcing. We detach no gradient between the streams in any variant.

A.6. Training Recipe

Table S2 lists the recipe for training on Dynamic-14. Every control step of a demonstration is a query window, which gives 1,324,374 training windows from the 11,200 demonstrations, and 350 held-out episodes serve as the validation split. Each loss is normalized by its count of valid labels over the whole accumulated global batch. The frozen networks run at every step: the image encoder processes the current frame, and the DINOv3 teacher's features of the future frames are cached once per episode. One training run takes about [X] hours on [N] RTX PRO 6000 GPUs. The DROID base uses the recipe of Appendix B.4.

Table S2. Training hyperparameters on Dynamic-14.

Item	Value
Backbone	24 layers, width 1024, 16 heads, FFN 4096, 504.0M parameters
Tokens	212 scene, 16 summary, 8 action
Encoder	frozen MAE ViT-B/16, 224×224 input
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Action noise	8 noise samples per window
Sampling	5 Euler steps for the action
Inference	FP16, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, about [X] hours per run

A.7. Deployment

Each query encodes the newest observation, runs the world stream once without its heads, and integrates the action stream on the cached world states. The first command executes as soon as the chunk is available, the policy observes again, and the next query starts once that command has executed. Until a new chunk arrives, the executor continues with the remaining commands of the previous chunk. Appendix C.3 describes the executor shared by all methods, and Appendix D.4 the timing measurements.

B. Pretraining on Real Robot Data

B.1. Data, Partitions, and Time

The pretraining corpus is DROID [36], recorded at 15 Hz with two exterior cameras and one wrist camera, together with the 3D point tracks and optimized camera calibrations that PointWorld released for it [30]. We associate each released clip with its raw DROID episode by requiring exact agreement of the joint sequence, end-effector poses, gripper states, commands, and camera identity, and we exclude clips whose capture clock is not monotonic or whose association is ambiguous. Of 42,935 associated episodes, 42,087 yield training windows. The model sees the first exterior camera, its depth, and PointWorld's calibration of it. The training unit is a causal observation window with its aligned future and action labels, one per control step, 598,889 in total. We assign episodes to training and validation partitions by their original identifier before any filtering, so the two share no episode. Table S3 reports the yield and the validity of the labels.

Table S3. **DROID pretraining data.** Yield of each stage and validity of the labels. Windows are causal observation windows with aligned labels, one per control step of the main camera. Flow labels count point–horizon slots.

	Train	Val.	All
Episodes associated	–	–	42,935
Episodes with windows	37,874	4,213	42,087
Windows	538,740	60,149	598,889
Valid actions (%)	—	—	96.88
Valid flow labels (%)	—	—	47.05
Future frames for DINO	—	—	3,425,218
Windows with moving objects (%)	—	—	—

Time. All times are capture timestamps, and we do not relabel frames to a nominal rate. The robot history uses the 8 most recent states at or before the query time, causally, as at deployment. Flow labels keep PointWorld's own record times rather than being interpolated to the 50 ms horizons of the simulation model. The future DINOv3 targets use the frames at the same record times.

Actions. We express robot states and Cartesian commands in the robot base frame after the calibrated flange-to-TCP transform, with the parameterization of Appendix A.4. We admit a label if its values are finite and its reference state was available causally. The robot's execution limits are not an admission rule, because a per-axis limit would discard fast but valid motion and bias the base toward slow demonstrations. Of 4,791,112 action slots, we mask the 149,318 at which DROID's collector issued no command and keep the remaining 96.88%, with no filter on magnitude or speed. Action and flow validity are independent, so an invalid action does not remove the flow labels of its window.

B.2. What the Base Learns

DROID provides no object poses, and its instructions name the manipulated object without marking it. The base therefore has no target-state tokens and no pose head. Its inputs are the current RGB-D frame and the robot history, and its objectives are the action loss, the sparse 3D flow of the patch center points (weight 1), and the future DINOv3 features (weight 0.1). Future depth, moving-region RGB, and the target reconstruction are off. Because PointWorld's tracks record only visible positions, the flow labels contain no negative visibility examples, and we set the visibility loss to zero rather than teach the model that every point stays visible. Most objects in DROID move only when the robot moves them, so the base mainly learns metric geometry, the 3D motion that the robot's own actions cause, and its coupling with the actions; the exogenous motion of our tasks and the target's pose enter during fine-tuning.

B.3. Flow Labels

PointWorld's tracks are already expressed in the base frame. Each patch of the 14×14 grid follows one fixed track over the window. Among the released tracks whose initial projection lies inside the patch, are initially visible, and have a valid positive depth, we prefer the one with the largest image motion and replace it by another candidate when its 3D displacement disagrees with its image motion. Otherwise we take the track nearest to the patch center. This rule covers moving objects that a fixed center point would miss: in a check of 30 segments from 10 episodes, 18 of which show visible object motion, it placed the start and two future points of a dynamic track inside the object in 13 of these 18 segments, against 5 for the center rule. This is coverage of image-consistent motion, not a measured 3D accuracy, and a patch on a small object can still drift to the background. The label is the track's displacement from its initial position at each record time. Patches without a valid track are invalid, and we do not fill them from neighbors. Over all windows, 47.05% of the point–horizon slots carry a valid label.

Label quality. We compared the PointWorld tracks with flow lifted from 2D correspondences [54] with monocular metric depth [75] on the same scenes. On surfaces that are static in the image, the selected PointWorld tracks move by [0.9] and [5.8] mm (median and 95th percentile) over the horizons, against [18] and [72] mm for flow lifted with monocular depth, whose scale drifts over time, so we use the PointWorld tracks and do not mix in the lifted flow. PointWorld's masks are not reliable enough to separate objects from the background, so we do not use them.

B.4. Recipe and Scale

Simulation model. SE(3)-WAM trains on Dynamic-14 from the generic visual initialization with the recipe of Table S2. We chose its depth among 12, 24, and 48 layers at width 1024 and its recipe with short runs at equal numbers of training samples, selected on development scenes (Fig. S2a).

DROID base. The base uses the same architecture family, with 48 layers at width 1024 (1.02B trainable parameters), and only real data. It trains from scratch for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch size of 512 on 64 GPUs, with the optimizer and precision of Table S2. The checkpoint with the lowest validation loss is the base (Fig. S2b). We train the controls of Table S4 at a pilot size of 12 layers and compare them with a world–action base of that size and budget.

B.5. Fine-Tuning and Controls

Each real-robot policy is initialized from the base and fine-tuned on the demonstrations of its task only. The target-state tokens, the pose head, and the appearance heads that the base lacks are added with fresh initialization, and the policy receives poses from the tracker of Appendix A.2. Simulation weights do not initialize any real-robot policy, and the base does not see simulation data. Each policy creates its own optimizer, schedule, training-only normalization, and readouts. The reference without the base is fine-tuned from the generic visual initialization on the same demonstrations, with the same search over [three] learning rates and two training lengths on development trials and the same evaluation. The controlled simulation comparison post-trains each base on Dynamic-14 with the downstream recipe and the same data fractions as the generic initialization. The action-only control pretrains the same model with every forecast loss disabled, and the other controls add the flow and DINO objectives in turn (Table S4).

Figure placeholder: Fig. S1 — flow labels from PointWorld tracks (schematic)

Left: one DROID frame of the main exterior camera with PointWorld's optimized camera pose next to the raw one, and the 14×14 patch grid. **Middle:** in each patch, the candidate PointWorld tracks (thin gray) and the selected track (orange): the one with the largest image motion among the tracks that stay consistent in 3D, falling back to the track nearest the patch center; patches without a valid track greyed out. **Right:** the selected tracks' 3D displacements in the base frame at PointWorld's record times, with the robot's gripper path in purple. Use one real DROID frame sequence, vector overlays only.

Figure S1. Flow labels for the DROID base. Each image patch follows one PointWorld track, selected to cover moving objects where they exist, and its 3D displacement in the base frame is the label. Invalid labels mask their losses.

Table S4. **Controlled comparison of initializations.** Each row fine-tunes the real-robot policy from its own initialization (SR over the five tasks, %) and post-trains the simulation model on 10/100% of the Dynamic-14 demonstrations (held-out SR, %, three post-training seeds). The last two columns give each base's held-out DROID errors: the flow endpoint error at the last horizon (mm) against PointWorld tracks and the action error of the first command (normalized). Control bases are pretrained at pilot scale and compared with a world–action base at the same budget (Appendix B.4).

	Real robot	Dynamic-14 SR, fraction of demonstrations		DROID val.	
Initialization	SR	10%	100%	flow (mm)	action
Generic visual only	—	—	—	–	–
Action-only base, pilot scale	—	—	—	–	—
Action + flow base, pilot scale	—	—	—	—	—
Action + flow + DINO base, pilot scale	—	—	—	—	—
Action + flow + DINO base, 48 layers (real-robot base)	—	—	—	—	—

Figure placeholder: Fig. S2 — recipe and scale

(a) Simulation model. y: development SR; x: 12, 24, and 48 layers at equal sample exposure; the selected configuration starred. **(b) DROID base.** x: training samples (log); y: validation flow endpoint error at the last horizon (left axis, mm) and action loss (right axis); thin lines for the pilot size, thick for the 48-layer run; marker at the selected checkpoint.

Figure S2. Recipe and scale of the simulation model (a) and the DROID base (b).

C. Dynamic-14

C.1. Tasks, Roles, and Terms

We built Dynamic-14 on MuJoCo and robosuite [72, 105]. It contains fourteen tasks in five families (Fig. S3). Table S5 lists their success conditions and Table S7 their scenario ranges. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Roles and critical leads. Table S6 assigns the bodies of each task to the two roles of Sec. 3.1. The target is the body whose pose error enters a contact or containment tolerance ε, and the context consists of bodies that only need to be avoided with clearance m. A moving receptacle is therefore a target, since its pose decides containment. The wire of the passage tasks plays both roles. Its pose at the entry is a target, because the wand must be aligned within 30 mm there, and its geometry along the loop is context, because any contact fails. The table also gives each task's speeds and its critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$ (Appendix G.5), and its last column lists the error terms that each task exercises (Eq. 3 and Appendices G.2–G.3). Conveyor and translating targets set the speed and the lead. Rotating targets, SE(3) placement, and the wire make orientation enter the contact. Ramps, low friction, and strikers produce motion that extrapolation misses, the occluder, wire, tray, and striker set the clearance, and the occluder also removes history.

We check success at every 2 ms physics step, after the ordered requirements of Table S5, and an interrupted continuous condition resets its counter. Terminal events fall into five categories that we report separately. An *invalid* event is a corrupted simulator state. *Boundary* events are a workspace or downstream exit, a drop, or an expired goal window. *Safety* events are unsafe contact, an excessive rim impulse, wire contact, a release impulse, early robot contact, unsupported tray contact, or a striker trap. *Task violations* are a release outside the goal, a path deviation or backtracking, a premature release, a deflection without capture, tipping, a

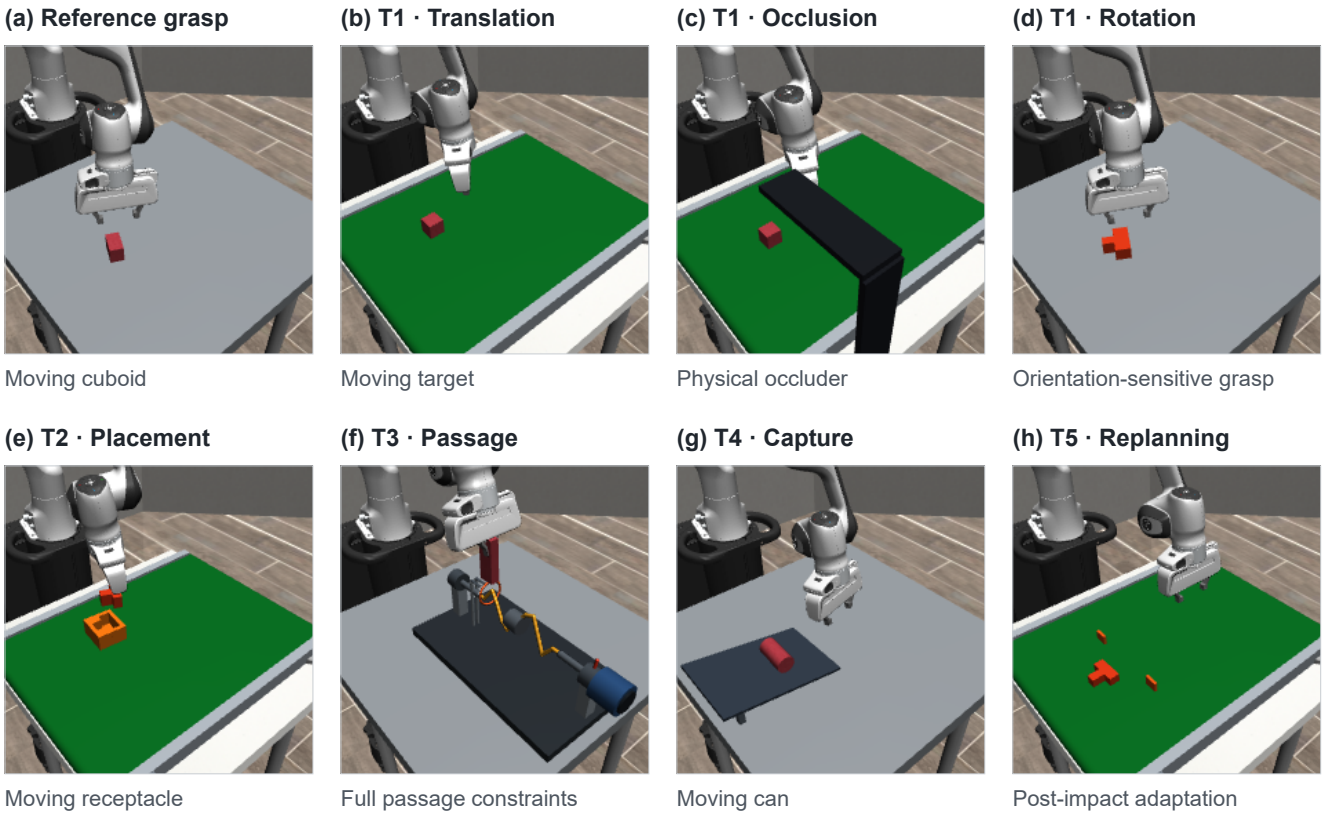


Figure S3. Dynamic-14 observations. Eight examples cover reference grasping, grasping a translating, occluded, or rotating target, placement into a moving receptacle, passage along a rotating wire, capture of a rolling object, and grasping after an impact. Each panel is the final observed, uncropped 224×224 agent-view RGB of one development scene. Table S5 lists all fourteen tasks.

missing or extra impact, or an insufficient change of outgoing motion. The last category is the *timeout*. Each family evaluates its events in a fixed order per physics step and keeps the first terminal event. The task-balanced score weights every task equally, including tasks on which some methods never succeed.

C.2. Scenarios and Demonstrations

We sample continuous ranges uniformly and discrete choices with equal probability, and then apply physical-feasibility checks. Evaluation replays the stored accepted scenarios. Harmonic translation follows $\mathbf{p}(t) = \mathbf{c} + (0, v_y, 0)t + (A_x \sin(2\pi f_x t + \phi), 0, 0)$ with $\phi \sim \mathcal{U}[-\pi, \pi]$. Capture targets start at rest and move under gravity and friction on ramps of $[5, 15]^\circ$. We accept an occluded scene only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames.

The training pool contains 11,200 demonstrations, 800 per task. They were generated by experts with privileged simulator state, whereas the learned policies receive only the inputs of Sec. 3.2. We record the demonstrations at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry. From these we compute the future labels, their validity masks, and the command labels. The policies receive the target pose from the simulator, which the pose-noise sweep perturbs (Appendix C.3).

C.3. Protocol and Sweeps

Executor. Every method runs under the same asynchronous executor on a 50 ms control clock. A method is queried again once one command has executed and its previous inference has finished, whichever is later, so a fast method replans after every executed command. While a query runs, the executor continues with the remainder of the previous chunk, one command per control period. A new chunk replaces this remainder and starts executing at the next control period. A baseline whose paper replans less often keeps its own chunk length under the *original executors* protocol.

Real-time protocol. Unless noted otherwise, physics advances during inference by a latency drawn per query from the method's measured latency distribution on one RTX 5090

Table S5. Success conditions of the fourteen Dynamic-14 tasks, with the thresholds of our task implementations. A task must pass its preceding stages and avoid terminal failures. All tasks run at 20 Hz with a 2 ms physics step; durations below refer to physical time. COM denotes center of mass.

Task	Ordered success conditions	Limit
grasp; t1_belt, t1_xyz, t1_so3, t1_occ	Approach within 0.10 m; bilateral fingerpad contact for 0.05 s; collision-geometry bottom at least 0.05 m above support; retain contact and clearance for 0.50 s. T1 adds conveyor drive, translation, rotation, or a physical occluder. SO3 has no additional success-angle threshold; unsafe occluder contact terminates OCC.	160 steps 8 s
t2_trans	Approach within 0.10 m. Entire object collision geometry inside the receptacle: planar clearance and floor gap both ≥ -0.002 m, top no higher than the opening. Release requires an opening command < -0.5 and no fingerpad contact. Relative linear/angular speeds ≤ 0.08 m/s and 0.8 rad/s for 0.10 s, followed by a 0.50 s stable dwell.	200 10 s
t2_se3	Approach within 0.05 m and 0.12 rad. Full T-shaped containment: planar clearance ≥ 0.002 m, floor gap in $[-0.002, 0.002]$ m, tilt ≤ 0.05 rad, top below opening. After release, relative speeds ≤ 0.02 m/s and 0.04 rad/s for 0.10 s, then a 0.50 s dwell. The 0.12 rad gate applies to approach.	200 10 s
t3_phase	The episode starts with the wand already grasped. Phase alignment: distance ≤ 0.090 m and loop-direction error ≤ 0.20 rad. Enter within 0.030 m at progress ≤ 0.15; reach historical maximum progress ≥ 0.60, then ≥ 0.85 and endpoint distance ≤ 0.055 m.	420 21 s
t3_full	Acquire the wand with bilateral contact for 0.05 s, then complete the phase stages above. In a later sample, retain current progress ≥ 0.85 and move at least 0.045 m from both path and endpoint. Finally open (< -0.5) with no fingerpad contact. Releasing before exit fails.	640 32 s
t4_sphere	Approach within 0.10 m; first qualifying contact after ramp exit; any tray contact before success fails. Bilateral hold for nominal 25 ms (12 physics samples, 24 ms). Raise COM by 0.05 m from its height when the bilateral-hold criterion is first met, and retain contact and height for 0.50 s.	120 6 s
t4_sphere_tabletop; t4_slider	Approach within 0.10 m; contact after ramp exit, while supported, in the capture region, and moving at planar speed ≥ 0.08 m/s. Bilateral hold for 0.05 s, COM lift ≥ 0.05 m from the first attainment of that hold criterion, then contact and height for 0.50 s.	120 6 s
t4_can	Same supported-capture sequence. Before first fingerpad contact, a can-axis absolute dot product with vertical > 0.45 fails as unstable tipping. The configuration does not require a transverse closing axis.	120 6 s
t5_replan	Exactly one selected-side impact group completes before robot contact. At 30 ms after the physical impact-end timestamp, branch-signed lateral speed ≥ 0.035 m/s and yaw rate ≥ 0.35 rad/s. Approach within 0.10 m; bilateral hold with grasp-axis error $\leq 18^\circ$ for 0.05 s; COM lift ≥ 0.05 m from the first attainment of that hold criterion and stable hold for 0.50 s. Impact-group lag is 12 ms and maximum contact is 0.10 s.	120 6 s

(Appendix D.4). The post-impact grasp additionally has a 50 ms command delay built into the task. Table S14 reports three alternatives. With *paused* physics, the simulation waits for inference, as in many simulated benchmarks. With *equal latency*, every method is charged the same latency, the p95 of SE(3)-WAM, regardless of its own speed. With *original executors*, each baseline keeps the execution scheme of its paper.

Latency. An added delay of 0, 50, 100, 150, or 200 ms is inserted after inference on top of the protocol's latency, and physics advances during it.

Speed. All exogenous velocities of a scene (belt, receptacle, wire, and the rate of harmonic motion) are multiplied by $s \in \{0.5, 1, 1.5\}$. Scenes that become physically infeasible at some factor are excluded for every method, and we report the remaining counts.

Clearance and grid resolution. The clearance sweep uses the passage tasks, whose wire is a context body. The free clearance between the wand's loop and the wire takes the values [X, X, X, X, X] mm, which bracket the spacings of the 14^2, 28^2, and 56^2 flow grids at the workspace ([X], [X], and [X] mm), while the object and the motion stay unchanged. Placement tolerances concern target precision (ε) and are not part of this sweep.

Pose noise. We add Gaussian noise to the target pose inputs, with magnitudes calibrated on the tracker's errors (Table S1), and we drop poses at the tracker's loss rate (Appendix F.2).

Table S6. **Roles, tolerances, and critical leads of the Dynamic-14 tasks.** The target is the body whose pose error enters a contact or containment tolerance ε. The context comprises bodies that must only be avoided, with clearance m. Speeds follow the sampling ranges of Table S7 (capture tasks: at the ramp exit), and the critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$ is given for the median speed. Terms: error terms that a task exercises (Eq. 3 and Appendices G.2–G.3). v: moving target. $r\theta$: orientation enters the contact. e_{dyn}: non-constant motion (harmonic, gravity, friction, impact). m: a context body must be avoided. Occ.: history gaps. Delay: command delay built into the task.

Task	Code	Target	Context	ε (mm)	$\|\mathbf{v}\|$ (m/s)	$\Delta^\star$ (ms)	m (mm)	Terms
Harmonic grasp	grasp	block	table	[X]	0.03–0.20	[X]	–	v, $r\theta$, e_{dyn}
Conveyor grasp	t1_belt	part	belt	[X]	0.04–0.10	[X]	–	v
Translating grasp	t1_xyz	part	table	[X]	0.04–0.15	[X]	–	v, e_{dyn}
Rotating grasp	t1_so3	part ($\omega \leq 1.2$ rad/s)	table	[X]	0.04–0.15	[X]	–	v, $r\theta$, e_{dyn}
Occluded grasp	t1_occ	part	occluder canopy	[X]	0.04–0.10	[X]	[X]	v, m, occ.
Moving receptacle	t2_trans	receptacle	table	[X]	0.03–0.06	[X]	–	v
Rotating receptacle	t2_se3	T-shaped receptacle	table	[X]	0.03–0.06	[X]	–	v, $r\theta$
Wire passage	t3_phase	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Wire passage, full	t3_full	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Ball capture	t4_sphere	ball	tray, ramp	[X]	[0.4–0.7]	[X]	[X]	v, e_{dyn}, m
Ball capture, table	t4_sphere_tabletop	ball	table, ramp	[X]	[0.4–0.7]	[X]	–	v, e_{dyn}
Slider capture	t4_slider	slider	table, ramp	[X]	[0.4–0.8]	[X]	–	v, e_{dyn}
Can capture	t4_can	can	table, ramp	[X]	[0.4–0.7]	[X]	–	v, $r\theta$, e_{dyn}
Post-impact grasp	t5_replan	part	striker	[X]	0.12–0.14	[X]	[X]	v, $r\theta$, e_{dyn}, m, delay

Occlusion and stationarity. Occlusions of 0.5, 1, 2, and 3 s and an unobstructed control isolate history gaps. Stationary-target variants freeze the exogenous motion of every task and serve as the static control of Sec. 4.4.

C.4. Splits and Statistics

We use a development pool of 1,400 scenes (100 per task) for every design decision. All Dynamic-14 numbers that we report, in the main paper and in this supplement, use a separate held-out set of 1,400 scenes, generated with the same ranges and with new source seeds that appear in no training, validation, or development episode. The first 100 feasible scenes in increasing seed order form each task's set.

The final model, its ablations, and the variants of Table 3 use training seeds 0, 1, and 2. The screening variants of Appendix F and the baselines use seed 0. On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR. Most of this spread came from the wire passage, whose success ranged from [0] to [71]% across seeds, and we do not interpret single-seed differences of a few points. Three seeds are too few to resample, so we average them per scene and bootstrap the held-out scenes within each task, jointly across methods, for paired 95% intervals. We treat differences whose interval lies within ± 2 points as ties.

C.5. Metrics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{1}{14}\sum_{k=1}^{14}\frac{1}{n_k}\sum_{i=1}^{n_k} y_{mki}, \qquad \text{(S3)}$$

for binary outcomes y_{mki} of method m on scene i of task k. The *feasible contact window* of a scene is the set of times at which a contact that satisfies the task's first success stage is kinematically possible. We compute it from the ground-truth target trajectory and the robot's reachable workspace. The *contact-time error* is the distance from the first-contact time to this window, and it is zero inside the window. The *position error at first contact* is the distance of the end effector from the target's grasp frame at that time. We report both on episodes that make contact. Failures without contact stay in the SR denominator, and their contact error is missing rather than zero. *Collisions* count safety events. *Time-to-action* is the wall-clock time from the arrival of an observation to the availability of the first command (Appendix D.4), and the *action age* is the execution time minus the capture time.

Table S7. Scenario proposal ranges. Length, linear velocity, angle, and angular velocity use m, m/s, rad, and rad/s unless marked otherwise. Positions are in world coordinates except the stated ramp-local offsets. Continuous intervals are uniform proposals; discrete choices are equiprobable before feasibility checks. Evaluation replays the stored accepted scenarios, whose distribution need not remain uniform. Harmonic centers are not initial positions.

Task	Initial-state proposal	Motion or event parameters
grasp	Harmonic center $c_x \in [-.08, .08]$, $c_y \in [-.26, -.19]$, $z = .822$; yaw $[-1.1, 1.1]$.	$v_y \in [.025, .125]$, $A_x \in [.015, .035]$, $f_x \in [.3, .7]$ Hz; $\omega_z \in [-1.2, 1.2]$.
t1_belt	Target $x \in [-.08, .08]$, $y \in [-.26, -.18]$, $z = .822$; identity orientation, zero initial twist.	Belt $v_y \in [.04, .10]$.
t1_occ	Same target proposal as Belt; canopy at $(.322, -.29, 1.20)$.	Same belt range; clearance parameter .20; physical occlusion.
t1_xyz	Harmonic center $c_x \in [-.025, .025]$, $c_y \in [-.24, -.20]$, $z = .818$; yaw $[-.35, .35]$.	$v_y \in [.04, .09]$, $A_x \in [0, .03]$, $f_x \in [.25, .65]$ Hz; $\omega_z = 0$.
t1_so3	Same as XYZ.	Same translation; $\omega_z \in [-1.2, -.4] \cup [.4, 1.2]$, equal sign probabilities.
t2_trans	Goal $x \in [-.13, -.08]$, $y \in [-.18, -.12]$, $z = .825$; fixed orientation.	Goal $v_y \in [.03, .06]$; no rotation.
t2_se3	Same goal x, y range, $z = .804$; yaw $[-.3, .3]$.	Goal $v_y \in [.03, .06]$; $\omega_z \in [-.25, -.08] \cup [.08, .25]$, equal sign probabilities.
t3_phase	Wand $x, y \in [-.03, .03]$, $z = .870$, yaw $[0, 2\pi]$; wire phase $[-.25, .35]$.	Wire center $(.10, -.02, .94)$; world-X rotation $\omega_x = -\{2.5, 3, \ldots, 7.5\}^\circ$/s (11 equal-probability values).
t3_full	Wand at $(-.05, -.02, 1.04)$ with identity orientation; same wire phase range.	Same wire rotation; cradle and release requirement enabled.
t4_sphere	Ramp-local offsets: lateral $[-.015, .015]$, longitudinal/normal $(-.145, .030)$; lip $(.06, -.02, 1.10)$.	Gravity-driven from rest; gate release at tick 0; slope $[5, 15]^\circ$; target/ramp sliding friction $[.7, .9]$; unsupported capture.
t4_sphere_tabletop	Lateral $[-.015, .015]$, longitudinal/normal $(-.124, .034)$; lip $(0, -.06, .808)$.	Same slope and release; target/ramp friction $[.7, .9]$, table friction .35.
t4_slider	Lateral $[-.015, .015]$, longitudinal/normal $(-.129, .024)$; supported ramp.	Same slope and release; target/ramp friction $[.025, .045]$, table friction $[.012, .018]$.
t4_can	Lateral $[-.010, .010]$, ramp-axis yaw $[-4, 4]^\circ$; longitudinal position follows gate geometry and oriented can extent.	Same slope and release; target/ramp friction $[.25, .45]$, table friction $[.09, .15]$.
t5_replan	$x \in [-.005, .005]$, $y = -.29$, $z = .812$; yaw $[-.08, .08]$; zero initial twist.	Belt $v_y \in [.12, .14]$; equal-probability left/right striker triggered at $y = -.13$; stroke .098, pulse .35 s, actuator scale $[1, 1.1]$, force cap 4 N; command delay 50 ms.

D. Baselines, External Benchmarks, Real Robot, and Timing

D.1. Baselines

Common conditions. Table S8 summarizes the adaptations. Each baseline keeps its official architecture, inputs, action chunk, and initialization. ACT and Diffusion Policy train from scratch, $\pi_{0.5}$ is fine-tuned from `pi05_base`, and the other methods start from their released weights. All of them train on the same 11,200 demonstrations, whose commands we map to each action space through the same Cartesian controller. Dynamic-14 renders a 224×224 wrist camera in addition to the agent view, and methods whose recipes use a wrist view receive it. SE(3)-WAM and the kinematic baselines use the agent view only. All methods run under the shared executor and the real-time protocol of Appendix C.3, and Table S14 adds their original executors.

Training. Each baseline follows its official recipe for the published number of updates and is trained once, with seed 0. We changed only the data loading and the action space. For DreamVLA, we ran both its original 40-epoch schedule and a 20-epoch cosine schedule and report the one that was better on development scenes. The configuration of SE(3)-WAM, in contrast, was selected by the ablations of Appendix F, a search that the baselines do not receive, and its variants use three training seeds.

Inputs. PUMA [19] receives its historical optical flow, computed on the rendered frames. DreamVLA receives

seven causal RGB frames, the end-effector position and orientation, the finger positions, and a fixed task text, and it samples with ten DDIM steps and classifier-free guidance of 1.5. OpenWAM-α receives the current RGB views, joint values, gripper opening, and task text, and it generates video and actions jointly with video decoding disabled. DynamicWAM receives its flow history and kinematic descriptors.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags and missing-state embedding. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. The output layer of the MLP is zero-initialized, so training starts from the behavior of the base model, which also provides the recipe.

Kinematic baselines. *SE(3)-WAM, CV input* shares the scene tokens and the action stream of SE(3)-WAM but has no forecast losses. Instead, the action stream receives the constant-velocity extrapolation of the target pose at the execution times of its chunk as input tokens, and the model is trained on the same data with the same recipe. Appendix F.3 adds a variant that receives these tokens and keeps the forecast losses. The *Kalman interceptor* has no learned components. A constant-acceleration Kalman filter [5] runs on the target pose, with its process noise tuned on development scenes, and predicts the target's trajectory. The interceptor chooses the earliest point of this trajectory that the arm can reach under its velocity limits after its measured latency, approaches it along the expert's pre-grasp offset, and closes, releases, or aligns at the predicted time. It replans at every query and uses the expert's task-specific grasp, placement, and entry frames.

D.2. DOMINO and LIBERO

DOMINO [19] contains 35 tasks of two interaction types, interception and tracking, at three dynamics levels: constant velocity, polynomial trajectories, and stochastic segments with abrupt changes. We use its [N] single-arm tasks on the Franka embodiment in the clean setting, at a speed coefficient of [0.1] and at all three levels, with the official demonstrations, the benchmark's own evaluator, and 100 closed-loop episodes per task and training seed. We report SR and DOMINO's manipulation score (MS), which scales route completion by penalties for leaving the workspace or the view and for collisions. On LIBERO [44], we use the Spatial, Object, Goal, and Long suites with ten tasks each, 50 evaluation episodes per task, three training seeds for our variants and one for baselines, and the released demonstrations and initial states. LIBERO checks that the role-aware forecast does not cost performance on static tasks. On both benchmarks, SE(3)-WAM trains from the same generic initialization as on Dynamic-14, with domain-specific normalization. As on Dynamic-14, it reads the pose of the object that the instruction acts on first from the simulator, and queries without a valid pose ([XX]% on LIBERO and [XX]% on DOMINO) use the missing-state embedding of Appendix A.1. The pose is privileged information that the baselines do not receive, so Table S9 also reports SE(3)-WAM with tracker-calibrated pose noise. SE(3)-WAM reaches [XX.X]% on DOMINO against [XX.X]% for [the strongest DOMINO baseline], and on LIBERO it is within [X.X] points of $\pi_{0.5}$.

Figure placeholder: Fig. S4 — real-robot setup

Left: photo of the cell: Franka arm with gripper, fixed third-person RGB-D camera, wrist camera, and the devices that move the targets ([belt conveyor, turntable, ramp]), with the base frame drawn. **Right:** one frame per task at the instant of the first command, annotated with the target's motion type (linear, circular, accelerating) and $\|\mathbf{v}\|\Delta$ for the measured latency.

Figure S4. Real-robot setup: the cell and the five tasks.

D.3. Real Robot

Platform. The robot is a seven-DoF Franka arm with a parallel-jaw gripper, commanded through a timestamped Cartesian interface at 20 Hz (Fig. S4). A fixed [RealSense D435] records RGB-D at [640 × 480] and [30] Hz, and we calibrate its extrinsics with a checkerboard before each session. SE(3)-WAM uses this third-person view, and $\pi_{0.5}$ also receives the image of a second camera on the wrist, as its recipe requires.

Tasks and success. The five tasks cover three kinds of target motion: linear motion ([T1, T2]), circular motion ([T3, T4]), and accelerating motion ([T5]). [For each task: the device that moves the target, the speed range, the objects, and the success condition, e.g., both fingers hold the object stably and lift it for at least 0.5 s.] Touching or pushing the target, a single-finger hold, and an action after the target has left the workspace count as failures. We measure the speed of every moving device at the start and end of every block, and trials whose speed deviates from the nominal value by more than 5% are repeated and reported as equipment failures.

Trials. Each method runs [20] trials per task. All methods use the same pre-generated trial keys, which fix the object instance, the speed, and the start pose, sampled from the calibrated reachable region. An external fixture that the pol-

Table S8. **Baseline adaptations.** All methods train on the same 11,200 demonstrations with the inputs of their papers and run under the shared executor of Appendix C.3. Updates × batch follows the official recipe. Exec.: the execution scheme of the original paper (commands executed per chunk before replanning), used only for the original-executor control of Table S14. Params: trainable parameters, frozen encoders excluded. Seeds: training seeds.

Method	Initialization	Inputs	Updates × batch	Exec.	Params	Seeds
ACT [99]	scratch	agent + wrist RGB, robot state	50,000 × 32	temporal agg.	[X]	1
Diffusion Policy [16]	scratch	agent + wrist RGB (published horizon), robot state	50,000 × 32	8	[X]	1
$\pi_{0.5}$ [57]	`pi05_base`	current agent + wrist RGB, robot state, task text	30,000 × 256	10	[X]	1
$\pi_{0.5}$ + SE(3)	`pi05_base`	as $\pi_{0.5}$ + SE(3) motion tokens	30,000 × 256	10	[X]	1
DynamicVLA [82]	released DOM ckpt.	frames $[-2, 0]$, robot state, task text	50,000 × 44	streaming	[X]	1
PUMA [19]	official checkpoint	RGB, optical-flow history, robot state, task text	[X]×[X]	[X]	[X]	1
DreamVLA [95]	released components	seven causal agent + wrist RGB frames, robot state, task text	48,220 × 512	1	[X]	1
OpenWAM-α [76]	official foundation	current agent + wrist RGB, joints, gripper, task text	34,490 × 192	32	[X]	1
DynamicWAM [50]	official initialization	RGB, flow history, kinematic descriptors	40,000 × 128	16	[X]	1
DynamicWAM + SE(3)	official initialization	as DynamicWAM + SE(3) motion tokens	40,000 × 128	16	[X]	1
SE(3)-WAM, CV input	as SE(3)-WAM	as SE(3)-WAM, plus constant-velocity forecast tokens, no forecast losses	as SE(3)-WAM	–	[X]	3
Kalman interceptor	none	SE(3) history, expert grasp frames	–	–	–	–
SE(3)-WAM	frozen MAE encoder	current agent RGB-D, SE(3) history, robot state	51,740 × 512	1	523.8M	3

Table S9. **External benchmarks** (three training seeds for our variants, one for baselines). DOMINO: SR (%) of the single-arm tasks by interaction type and by dynamics level (L1 constant velocity, L2 polynomial, L3 stochastic with abrupt changes), and the manipulation score MS. LIBERO: SR (%) per suite.

	DOMINO						LIBERO				
Method	Intercept	Track	L1	L2	L3	MS	Spatial	Object	Goal	Long	Mean
ACT	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—	—	—	—	—	—
PUMA	—	—	—	—	—	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, noisy poses	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—	—	—	—

icy does not see sets and records the start pose. Each task's trials run in random order, so that drift of the apparatus does not align with the methods. An operator resets the scene by hand. A trial starts at the first usable observation, without a free observation period, and the robot may move at any time. Human contact, safety stops, and manual completion count as failures. Infrastructure failures before the first usable observation are repeated and reported. Test trials use object instances that appear in no demonstration, and demonstrations, development trials, and test trials come from separate sessions.

Fine-tuning. SE(3)-WAM is fine-tuned from the DROID base on [N] teleoperated demonstrations per task (Appendix B.5). The reference without the base starts from the generic initialization with the same demonstrations and search, and $\pi_{0.5}$ and DynamicWAM are fine-tuned from their released weights on the same demonstrations. The tracker uses [a CAD model or reference views of each object] (Table S1).

Analysis. We measure the contact-time error against the feasible contact window computed from the device's encoder,

and failures without contact remain in the SR denominator. Table S10 reports successes per task with 95% binomial intervals for the pooled rates and paired differences from a bootstrap over trial keys. The evaluation took [X] days. Most failures of SE(3)-WAM are [late closures after tracking losses under the gripper].

D.4. Timing

We time all methods on one RTX 5090 (32 GB), the robot's deployment GPU. For each method, timing uses thirty warm-up calls and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and three rounds with rotated method order, and each method uses the precision of its release. The time-to-action includes encoding the image and generating the first command, and on the robot also tracking the newest frame. For SE(3)-WAM, we record the time of the forecast heads separately, although deployment skips them. Table 1 gives the p95 values and Table S8 the parameter counts. On the robot, traces record capture, availability, preprocessing, the start and end of inference, queue entry, command issue, and execution. The action age is the execution time minus the capture time, and we report its p50, p95, and maximum together with the fraction of missed deadlines and the peak memory (Table S11). The latency sweep of Dynamic-14 differs from wall-clock deployment, because it inserts a prescribed delay while physics advances.

Table S10. **Real-robot successes** per task (out of [20] trials each), grouped by the target's motion. All: success rate (%) over all trials of a method with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys. e_t: median contact-time error (ms) on trials that make contact.

	Linear		Circular		Accel.			
Method	[T1]	[T2]	[T3]	[T4]	[T5]	All	Δ	e_t
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
SE(3)-WAM, no DROID base	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	–	—

Table S11. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Deadline misses (robot, %)	—	

E. Additional Results

E.1. Per-Task Results and Outcomes

Table S12 breaks the held-out Dynamic-14 results of Table 1 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task. Table S13 reports how episodes end. A method that trades one failure for another, e.g., fewer collisions for more timeouts, shows this only here. Every failure stays in the success-rate denominator.

E.2. Protocols and Latency

Table S14 evaluates every method under three further protocols. Paused physics removes the cost of computation. Equal latency charges every method the p95 latency of SE(3)-WAM and so separates a method's speed from the way it uses time. The original executors test whether our shared executor puts any method at a disadvantage. With paused physics, the difference between SE(3)-WAM and [DreamVLA] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S15 gives success at every added latency of the sweep, and its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

E.3. Negative and Null Results

Visual histories. None of the eight visual histories of Table S17 improved on the current frame by more than two points. The intervals of [N] of them lay within ±2 points, and [N] were inconclusive, including the RGB-D window with metric positions. Frozen readouts recover past target positions from these histories with [X]–[X] mm error, against [X] mm for the SE(3) history. The information is therefore present, but at our data scale the policy does not turn it into metric motion. A larger dataset or a dedicated motion objective might change this.

Image-space futures. As the forecast, no image-space future of Table S20 (scene tracks, dense displacement, codec vectors, video latents, a frozen video model, depth, DINO, or dynamic RGB) exceeded the target's SE(3) trajectory without context flow by more than [X.X] points. Their objectives weight pixels or latent dimensions, while the tolerance depends on metric error. Added next to the role-aware forecast as training-only targets, depth, DINO features, and moving-region RGB gained [X.X] points. Appearance therefore helps as an auxiliary target, although it does not replace the metric forecast.

Context resolution. Above the path error of the coarse flow, the 28^2 and 56^2 grids and the object-sampled flow matched the 14^2 grid within two points at up to [X]× its cost. Their advantage is confined to the tightest clearances.

Coupling. No variant in which the actions read world queries (R2), the world reads the actions (R3), both (R4), or the actions read the decoded forecast (R5) improved consistently over SE(3)-WAM at any latency (Table S23), and each raised the time-to-action by [X.X] to [X.X] ms. Disabling the forecast losses in the same network (R0) cost [X.X] points, so the gain comes from the supervision of what the summary tokens represent and not from reading the forecast.

Unsolved tasks. The SE(3) placement into the rotating receptacle (t2_se3) fails for every method (Table S12). [The policies reach the receptacle but miss its angular tolerance before it turns away.] No variant of SE(3)-WAM solves the full wire passage (t3_full), which adds grasping and releasing the wand to the passage, whereas [OpenWAM-α] succeeds

Table S12. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. CV: SE(3)-WAM with CV input. KF: Kalman interceptor. σ: SE(3)-WAM with tracker-calibrated pose noise. Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. Task codes are listed in Table S6.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	CV	KF	σ	**Ours**	Δ [95% CI]
Harmonic grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Slider capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Can capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Table S13. **Outcome categories** on held-out Dynamic-14 scenes (% of episodes, seed-averaged, real-time protocol). Categories follow Appendix C.1, and rows sum to 100.

Method	Success	Safety	Task	Bound.	Timeout	Inval.
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
Kalman interceptor	—	—	—	—	—	—
SE(3)-WAM, noisy poses	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

Table S14. **Protocol controls** on held-out Dynamic-14 scenes (SR, %). Paused: physics stops during inference, and each method uses the execution scheme of its paper. Equal latency: shared executor, with every method charged the p95 latency of SE(3)-WAM. Original executor: real time with each method's own execution scheme. Real time: the protocol of Table 1. Dropped: commands discarded as stale in real time (%).

Method	Paused	Equal latency	Original executor	Real time	Dropped (%)
ACT	—	—	—	—	—
Diffusion Policy	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM, CV input	—	—	–	—	—
Kalman interceptor	—	—	–	—	—
SE(3)-WAM, noisy poses	—	—	–	—	—
SE(3)-WAM	—	—	–	—	—

in [XX]% of these episodes. On the passage that starts with the wand grasped (t3_phase), the success of SE(3)-WAM ranges from [0] to [71]% across training seeds.

Other failures. The collapse of Fig. S6a is weakest for the capture tasks (Spearman $\rho = [X.XX]$), where success also depends on when the gripper closes, and the noise crossover lies [X.X] times above the model-free $\sigma^\star$. Post-impact grasping gains least from the forecast losses ([X.X] points over the CV input), since a conditional mean cannot represent the outcomes of an impact. Tracker-calibrated pose noise costs [X.X] points, mostly on symmetric and thin targets. Most remaining failures of SE(3)-WAM are [e.g., timeouts after a missed first grasp] (Table S13).

Figure placeholder: Fig. S5 — forecasts aligned with what happened

Rows 1–4: the median-error held-out episode of each motion type (conveyor, rotating target, rolling capture, post-impact); **row 5:** a failure. **Columns:** the query frame at τ_t; the frame at the first command's execution time τ^{a}; and a 3D inset in the base frame. **Overlays:** blue past SE(3) poses; orange predicted target triads at $\tau_t + \delta_h$ and sparse flow arrows, read from the training-time heads; gray ground-truth poses at the same physical times; purple issued gripper path; a marker at first contact. A strip under each row plots translation and rotation error vs. horizon for learned SE(3), constant velocity, and constant acceleration. Annotate the failure cause of row 5 in red.

Figure S5. Forecasts of the training-time heads on held-out episodes, aligned in physical time with the ground truth and the issued actions. Rows 1–4 show the median-error episode of each motion type, and row 5 shows a failure.

Table S15. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the All and +150 ms columns of Table 1. Slope: least-squares change of SR per 100 ms of added latency (points).

	Added latency (ms)					
Method	0	50	100	150	200	Slope
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
Kalman interceptor	—	—	—	—	—	—
SE(3)-WAM, noisy poses	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

E.4. Forecasts at Execution Time

Fig. S5 aligns the query, the forecast, the ground truth, the issued actions, and the first contact on physical timestamps. The examples show where forecast errors arise—e.g., at the moment of an impact or when a rolling object leaves its ramp—and how they turn into contact-time and pose errors.

E.5. Supplementary Video

The supplementary video shows real-robot trials of SE(3)-WAM and the strongest baseline on all five tasks with the tracked poses overlaid, a set of Dynamic-14 episodes per task family with the forecasts of the training-time heads, and the failure cases discussed in Sec. 4.3.

F. The Complete Ablation Study

This appendix lists every variant we compared while designing SE(3)-WAM, including those not shown in Table 3. Every comparison varies one axis, keeps the others at the selected configuration, and uses the same scenes and inference seeds for every variant. We screened the variants at a reduced scale, with 24-layer models replaced by 4 layers of width 768 trained for 4 epochs (40,104 updates at a global batch size of 16), with one training seed, and made every selection decision on the development pool. The rows of Table 3 use the final model and recipe (Appendix A.6). The tables report the held-out scenes. The encoder and coupling comparisons and the rows that also appear in Table 3 average three training seeds, and the other rows use one. In total, the study took about [XXXX] GPU-hours of training and [XXXXXX] evaluation episodes.

Selection. A candidate had to keep the p95 time-to-action below 200 ms and its peak memory below 80% of one RTX 5090 at deployment precision. Because single seeds differ by several points (Appendix C.4), we replaced the reference only when a candidate gained more than 3 points on development scenes with seed 0 and kept a gain with seeds 1 and 2. Otherwise we kept the cheaper or the simpler variant.

F.1. Shared Components

The observation encoder and the training-only appearance targets are shared by all comparisons, and we fixed them first. We compared a CNN trained with the policy with frozen DINOv3 ViT-S/16 and ViT-L/16 [64] and MAE ViT-B/16 [26] encoders on development scenes (Table S16). The frozen MAE encoder had the best mean of three seeds, [66.8]%, but its seeds ranged from [63.7] to [71.1]% and overlapped with the DINOv3 encoders. We chose it for this mean and because it costs less than ViT-L/16. DINOv3 ViT-L/16 features re-

Table S16. **Observation encoder** on held-out scenes (three training seeds), with every other component at the selected configuration (⋆: used). Params: parameters of the encoder. p95: time-to-action on one RTX 5090.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16⋆	86M	frozen	—	—	—

main one of the future appearance targets, which we selected together with the world representation (Appendix F.3).

F.2. History

Conditions. Every condition receives the current image, the *current* target pose, eight robot states, the selected forecast targets, and the selected coupling. The conditions differ only in what they keep from the past 350 ms. The RGB window keeps seven past frames of 196 patches each, sampled every 50 ms, without compression. Optical flow is bidirectional Farnebäck flow [20] on 64×64 RGB over four intervals of 100, 100, 100, and 50 ms that span the same 350 ms window, with forward–backward validity, and "flow + motion statistics" appends the twelve physical statistics per interval of DynamicWAM [50]. Codec vectors come from actual MPEG-4 P-frame motion vectors and are encoded like the flows, and the video latent is the posterior mean of the frozen Wan2.2 VAE, read by 56 learned queries. The compressed windows read past frames with a shared bank of eight queries, either uniformly or with more queries for newer frames, following the allocation of PACT-WAM [48]. The RGB-D window adds the metric position of every past patch and tests whether the missing ingredient is metric geometry rather than motion. Among the SE(3) variants, the position-only history removes orientation, and the noisy history perturbs the past poses with the tracker-calibrated noise of the pose-noise sweep below. A final control removes the current target pose from the RGB window to measure what the current pose alone contributes.

Frozen readouts. To separate the information that a history encoding *contains* from the information that the policy *uses*, we fit ridge readouts on frozen features at three sites, the output of the history encoder, the final target-state tokens, and the input of the action head at the first function evaluation. The labels are the target position 50, 200, and 350 ms in the past, the velocity over the last interval, and the future displacement at 400 ms, all in the base frame. Each task assigns 60, 20, and 20 distinct development episodes to training, validation, and testing, with five windows per episode. We choose the penalty from $\{0.1, 1, 10, 100\}$ on the validation error, and a shuffled-label control bounds chance performance. Conditional readouts first fit a baseline from the current image, geometry, robot state, and current pose and then predict its residual from the site features, which tests what the history adds beyond the present.

Perturbations and interventions. Offline perturbations repeat the current image in all past slots or reverse the past images while keeping the current frame, the poses, and the noise fixed, and we measure the change of the first issued action. On the final model (Table S18), *stationary history* replaces every past pose with the current pose before the kinematics are recomputed, which removes displacement, velocity, and acceleration. *Time reversal* reverses the past poses. A *matched-current swap* substitutes the history of the nearest different development episode with the same task, object, and camera and with current target and end-effector poses within 5 mm and 5°. The *direction-ambiguous* subset additionally requires opposite last-interval velocities of at least 0.01 m/s.

Pose noise. Simulator poses are exact, so we test the sensitivity to tracking errors directly. We add Gaussian noise with standard deviation $\sigma_p \in \{0, 1, 2, 5, 10, 20, 40\}$ mm in position and $\sigma_R = \sigma_p/(30\,\mathrm{mm})$ rad in rotation, so that both produce the same displacement at a 3 cm lever arm, independently to every past pose estimate before the kinematics are computed. The current pose is unchanged, so the test isolates the history. We compute the predicted crossover $\sigma^\star$ (Appendix G.5) per task from Table S6 with $T = 350$ ms, $n = 8$, and the realized leads, and mark it in Fig. S6b together with the tracker's measured error (Table S1). The "tracker noise" rows use the noise level that matches this error, together with pose dropouts at the tracker's loss rate.

Occlusion and sampling. Occlusions of 0.5, 1, 2, and 3 s and an unobstructed control test history gaps. The sampling tests keep the 350 ms span and select 2, 4, or 8 offsets, with time embeddings at the actual offsets.

F.3. Forecast

Representations. Table S19 defines the families of future targets. All targets are cumulative from the current state and cover the same eight horizons from 50 to 400 ms. Every row of Table S20 keeps the history, coupling, action loss, training budget, and world-stream capacity of the selected configuration and replaces only the forecast targets, which readout heads regress from the world stream as in Appendix A.3. We therefore compare image-space targets at matched capacity. The frozen-video row instead reads latents that a pretrained image-to-video model predicts from the current frame, a future that does not depend on our capacity. The Kabsch row predicts the object tracks and fits SE(3) to them, so it receives the same information as the track row plus a rigidity

Table S17. **History: complete comparison** on held-out scenes (one training seed, three for rows that also appear in Table 3). All conditions keep the current image, the current target pose (except the last row), robot history, the selected future representation, and the selected coupling. SR (%) under added latency. Readout: frozen-ridge error of the target position 350 ms in the past (mm, lower is better). δa: change of the first issued translation when past inputs are repeated (mm). Tokens: added history tokens. p95: time-to-action (ms).

	SR at added latency (ms)								
History	0	50	100	150	200	Readout	δa	Tokens	p95
Current frame only	—	—	—	—	—	—	0	0	—
RGB window	—	—	—	—	—	—	—	1,372	—
RGB-D window	—	—	—	—	—	—	—	1,372	—
Optical flow	—	—	—	—	—	—	—	60	—
Flow + motion statistics	—	—	—	—	—	—	—	60	—
Codec motion vectors	—	—	—	—	—	—	—	56	—
Video-VAE latent	—	—	—	—	—	—	—	56	—
Compressed window (age)	—	—	—	—	—	—	—	56	—
Compressed window (uniform)	—	—	—	—	—	—	—	56	—
SE(3) history*	—	—	—	—	—	—	—	8	—
SE(3) history, position only	—	—	—	—	—	—	—	8	—
SE(3) history, tracker noise	—	—	—	—	—	—	—	8	—
SE(3) history + RGB window	—	—	—	—	—	—	—	1,380	—
RGB window, no current pose	—	—	—	—	—	—	—	1,371	—

Table S18. **Frozen-policy interventions.** Each row changes one input at inference only. History interventions use the final model. Hidden-state interventions use coupling R2, and geometry interventions use R5 (the final model reads no forecast). SR (%) on held-out scenes. δa: mean change of the first issued translation (mm).

Input	Intervention	SR	δa
History	none	—	0
	stationary (current pose repeated)	—	—
	time-reversed	—	—
	matched-current swap	—	—
	direction-ambiguous swap	—	—
World states (R2)	none	—	0
	matched-current swap	—	—
	horizon times shifted ± 50 ms	—	—
Decoded geometry (R5)	none	—	0
	zero motion	—	—
	matched-current shuffle	—	—
	time shift ± 50 ms	—	—
	oracle (pre-contact)	—	—

prior. The last block keeps the selected forecast and varies only the training-only appearance targets, whose heads we drop at deployment, so it measures what these targets teach the world stream.

On development scenes, most representations stayed within a few points of each other overall, and their differences concentrated in a few tasks. Target point tracks with the flow even reached a slightly higher overall SR than the SE(3) trajectory ([+1.7] points with one seed). We kept the SE(3) trajectory, which gives the pose directly and which matters most for rotating targets, and Table S20 compares both on the held-out scenes. Dense 2D displacement was the only future that clearly hurt ([−3.6] points against no future).

Role coverage. On the selected representation, the role rows keep all source points and the same computation and change only which points, those on the target or those on the context, contribute to the flow loss.

Resolution and clearance. The 28^2 and 56^2 grids sample the same image more densely, and the object-sampled flow places its points on the context masks instead of a fixed grid. All variants use identical labels, normalization, and loss weights. The path error e_F is the endpoint error of the flow at the 256 held-out context points nearest to the executed path, at the times when the path passes them. Table S21 reports all variants on the clearance sweep of Appendix C.3, together with a video-latent future at matched capacity.

Extrapolation baselines. Constant-velocity and constant-acceleration forecasts use only the causal SE(3) history visible to the model, i.e., the velocity of the last interval and the difference of the last two interval velocities, applied to translation and, through the rotation vector, to rotation. Offline, we score all three forecasts against the ground truth at the execution times. To measure their closed-loop value,

Table S19. Future-target families compared in the *Predict* study. Shape excludes batch and query axes. Displacements are cumulative from the current state, except for codec vectors, which keep the correspondence of the decoder. All targets come from the recorded demonstrations except the frozen video model, whose latents are its own predictions. Future depth, DINO, and dynamic RGB are also the training-only auxiliary targets of the selected model.

Target	Shape	Support, units, and validity
Target pose	8×6	Cumulative translation (m) and left-relative rotation vector (rad). Requires a valid pose and query.
Scene flow, $k \times k$ grid	$k^2 \times 8 \times 3$	Fixed grid of $k \in \{14, 28, 56\}$ points per side, in base-frame meters. Requires valid source depth and a finite future displacement.
Scene flow, object-sampled	$196 \times 8 \times 3$	Points sampled on the context masks by farthest-point sampling instead of a fixed grid.
Object tracks	$32 \times 8 \times 3$	Farthest-point samples in the current target mask, with displacement and visibility. Padded points are invalid. The Kabsch variant fits SE(3) to the predicted tracks.
Object DINO	8×1024	Mask-weighted pooling of future patch features with channel layer normalization. Requires a positive mask area.
Scene tracks (2D)	$196 \times 8 \times 3$	Grid source points with two image displacements divided by 224 and a visibility logit.
Dense 2D displacement	$8 \times 2 \times 224 \times 224$	Fixed source pixels, displacement divided by 224. Requires a valid source and a finite projection.
Codec vectors	$8 \times 2 \times 64 \times 64$	Each future/current pair is encoded as P/I frames, and vectors are divided by 64. Only valid vector regions count.
Video-VAE latent	$48 \times 2 \times 14 \times 14$	Frozen Wan2.2 VAE posterior mean with the VAE's own normalization. The two groups end at 200 and 400 ms.
Frozen video model	$48 \times 2 \times 14 \times 14$	Latents that a frozen pretrained image-to-video model predicts from the current frame, read as input tokens without supervision.
Future depth	$8 \times 1 \times 224 \times 224$	Metric depth on finite positive pixels.
Future DINO	$8 \times 196 \times 1024$	Frozen DINOv3 ViT-L/16 teacher [64], 14×14 patches, channel layer normalization.
Dynamic RGB	$8 \times 3 \times 224 \times 224$	RGB divided by 255 on surfaces moving more than 1 mm, including the robot.

we train two variants with the same data and budget (Table S22). The first receives the extrapolated target trajectory as additional input tokens and keeps the forecast losses of the selected model. The second replaces the forecast losses by these input tokens and is the CV-input baseline of Table 1.

Forecast errors. Physical errors use a common pool of 256 target and 256 context source points, sampled deterministically and excluded from the training queries. Translation and rotation errors keep their metric and geodesic units, and flow errors are endpoint errors on valid correspondences. All forecast errors are measured on held-out episodes at the execution times.

F.4. Use of the Forecast and Backbone

Backbones. The shared-attention backbone shares the attention projections across all token groups and splits the feed-forward networks by stream. The separate-group backbone has world and action parameter groups with their own attention projections, adaptive layer normalization, and feed-forward networks. We match the number of trainable parameters through the feed-forward ratio of the separate-group backbone (3 instead of 4), which left both at about 47.3M parameters at the screening scale, with the same depth, width, and number of heads. Equal training budgets do not imply equal FLOPs, so we report the measured costs. On development scenes, the shared-attention backbone reached [66.8]% against [62.9]% (three seeds each), and we kept it.

Couplings. Appendix A.5 defines the attention masks of R0–R5. All couplings share the modules, initialization, losses, and training budget. R0 keeps the readout heads but disables every forecast loss, so it matches the capacity of SE(3)-WAM without forecast supervision. R1 adds world queries that carry the readouts but opens no edge between them and the actions, so it differs from SE(3)-WAM only by these queries. We deploy SE(3)-WAM both action-only and with the forecast heads computed but not read, which isolates the cost of an unused forecast. R2 lets the action read the hidden states of the world queries, R3 lets the world queries read the noisy actions, and R4 opens both directions. R3 and R4 recompute the world at every Euler step. R5 decodes the forecast and re-encodes it for the action with a geometry encoder of 0.37M parameters, trained end to end through the predicted geometry without teacher forcing. Because R5 changes both what the action reads and where it enters, the comparison of R2 and R5 is between complete designs rather than one factor.

On development scenes, no coupling beat R1 consistently. Letting the world read the action (R3) had the highest mean

Table S20. **Forecast: complete comparison of future representations** on held-out scenes (one training seed, three for rows that also appear in Table 3). History and coupling are fixed at the selected configuration, the world stream has the same capacity in every row, and only the world targets change (definitions in Table S19). Rows above the last block keep the selected training-only appearance targets. The last block varies these targets on the selected representation, with their heads dropped at deployment. Pose: translation (mm) and rotation (°) error at 400 ms. e_F: path error of the flow on held-out context points along executed paths (mm). e_t: contact-time error (ms). Cost: world-stream time per query (ms).

Target	Context	SR	SR_{150}	Pose (mm/°)	e_F	e_t	Collision	Cost
none	none	—	—	–	–	—	—	0
SE(3)	none	—	—	—	–	—	—	—
none	flow, 14^2	—	—	–	—	—	—	—
SE(3)	flow, $14^{2\star}$	—	—	—	—	—	—	—
SE(3)	flow, 28^2	—	—	—	—	—	—	—
SE(3)	flow, 56^2	—	—	—	—	—	—	—
SE(3)	flow, object-sampled	—	—	—	—	—	—	—
object tracks	flow, 14^2	—	—	–	—	—	—	—
tracks → SE(3) fit	flow, 14^2	—	—	—	—	—	—	—
object DINO	flow, 14^2	—	—	–	—	—	—	—
SE(3)	scene tracks (2D)	—	—	—	–	—	—	—
SE(3)	dense 2D displacement	—	—	—	–	—	—	—
SE(3)	codec motion vectors	—	—	—	–	—	—	—
SE(3)	video-VAE latent	—	—	—	–	—	—	—
SE(3)	frozen video model	—	—	—	–	—	—	—
SE(3)	future depth	—	—	—	–	—	—	—
SE(3)	future DINO	—	—	—	–	—	—	—
SE(3)	dynamic RGB	—	—	—	–	—	—	—
Role coverage on the selected representation								
SE(3)	flow, target points only	—	—	—	—	—	—	—
SE(3)	flow, context points only	—	—	—	—	—	—	—
Training-only appearance targets on the selected representation (SE(3) + flow)								
none		—	—	—	—	—	—	—
future depth		—	—	—	—	—	—	—
future DINO features		—	—	—	—	—	—	—
moving-region RGB		—	—	—	—	—	—	—
all three*		—	—	—	—	—	—	—
all three, no current-target RGB reconstruction		—	—	—	—	—	—	—

Table S21. **Context resolution versus clearance** on the passage tasks: grid spacing at the workspace and path error e_F (mm), and SR (%) at the swept clearances $c_1 < \cdots < c_5$ (Appendix C.3). The target is predicted in SE(3) throughout.

Context	Spacing	e_F	c_1	c_2	c_3	c_4	c_5
none	–	–	—	—	—	—	—
flow, 14^2	—	—	—	—	—	—	—
flow, 28^2	—	—	—	—	—	—	—
flow, 56^2	—	—	—	—	—	—	—
flow, object-sampled	—	—	—	—	—	—	—
video-VAE latent	–	–	—	—	—	—	—

of three seeds, [+0.6] points over R1, but the whole difference came from the wire passage, and the effects of both directions of access changed sign across seeds. Generating the action from the decoded forecast (R5) did not improve on R1 either. We therefore kept the training-only use of the forecast without world queries, which is also the cheapest to deploy. The forecast supervision itself, R1 against R0, added [3.1] points on average, but its gain ranged from [−1.3] to [+9.6] points across seeds.

Forecast interventions. The interventions hold the scene and summary tokens and the action noise fixed and change only what the action reads from the world. On R2, they replace the keys and values that the action tokens read at every layer. A matched-current swap substitutes the world states of a different development episode whose current target pose is within 5 mm and 5°, and a time shift recomputes them

Table S22. **Forecast losses versus extrapolated inputs**, by motion type. Top: SR (%) of variants of SE(3)-WAM, trained with the same data and budget, that receive the constant-velocity (CV) or constant-acceleration (CA) extrapolation of the SE(3) history as input tokens, with or without the forecast losses. The CV row without forecast losses is the CV-input baseline of Table 1. Bottom: target position error at 400 ms (mm) of the two extrapolations and of the learned pose head.

	Const. vel.	Harmonic	Roll/slide	Impact
Without forecast losses				
no forecast input	—	—	—	—
CV input	—	—	—	—
CA input	—	—	—	—
With forecast losses				
SE(3)-WAM (no input)	—	—	—	—
CV input	—	—	—	—
CA input	—	—	—	—
Position error at 400 ms (mm)				
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
learned pose head	—	—	—	—

with every horizon offset shifted by 50 ms. On R5, they edit the decoded geometry before it is encoded. Zero motion uses zero 3D displacement and identity target poses with unchanged visibility, a matched-current shuffle takes the geometry of another development episode under the same matching, time shifts resample the trajectory at $\delta_h \pm 50$ ms, and the oracle uses the measured exogenous motion before the first contact. After contact, future labels depend on the action, so the oracle analysis forks a simulator snapshot under a fixed sequence of candidate actions. SE(3)-WAM, whose actions read no forecast, serves as a negative control.

Table S23. **Use of the forecast: complete comparison** on held-out scenes. History and forecast targets are fixed at the selected configuration. The couplings of Appendix A.5 are compared on the selected backbone, and the other backbone is checked with two of them. $\mathrm{SR}_{\mathrm{static}}$: stationary-target variants. NFE: network evaluations per query (world/action). Memory: peak GPU memory (GiB). p95 on one RTX 5090.

Backbone	Coupling	SR	SR_{150}	$\mathrm{SR}_{\mathrm{static}}$	LIBERO	NFE	p95 (ms)	Memory	e_t
Shared attention*	SE(3)-WAM (readouts, no edge)*	—	—	—	—	1/5	—	—	—
	SE(3)-WAM, forecast heads run	—	—	—	—	1/5	—	—	—
	R0 (forecast losses off)	—	—	—	—	1/5	—	—	—
	R1 (world queries, no edge)	—	—	—	—	1/5	—	—	—
	R2 (world→action, hidden)	—	—	—	—	1/5	—	—	—
	R3 (action→world)	—	—	—	—	5/5	—	—	—
	R4 (bidirectional)	—	—	—	—	5/5	—	—	—
	R5 (decoded geometry)	—	—	—	—	1/5	—	—	—
Separate groups	SE(3)-WAM	—	—	—	—	1/5	—	—	—
	R2 (world→action, hidden)	—	—	—	—	1/5	—	—	—

G. Error Analysis of Acting Under Delay

This appendix derives the error decomposition of Sec. 3.1, extends it to rotation, the context, and chunked execution, derives simple scaling relations, gives magnitudes for Dynamic-14, and states where it does not apply. The decomposition motivates the design of SE(3)-WAM and is not a model of success.

G.1. Translation

Let the true target position at the execution time be $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$ and the policy's estimate $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives the identity

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S4}$$

and the triangle inequality yields Eq. 3. Since $\mathbf{v}$ is the velocity at τ_t, $\boldsymbol{\rho}$ is exactly the part of the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, and an acceleration estimate from the history removes part of e_{dyn}. Impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce. For a learned predictor, the split of its estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$ is not identified, because any predicted position can be written in this form. We therefore use the decomposition to say which information each term requires. Our experiments do not separate the terms inside a learned predictor. Instead, they vary the lead, the speed, the pose noise, and the timing, which act on known terms, and they compare variants that differ in the information they receive (Appendix G.5).

G.2. Rotation and the Contact Point

We write $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$ and $\hat{R}(\tau^{\mathrm{a}}) = \mathrm{Exp}(\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}})\hat{R}(\tau_t)$. When R and $\hat{R}$ rotate about a common fixed axis, the exponentials compose additively, and

$$\theta = \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^\top)^\vee \big\| \le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta, \tag{S5}$$

where θ_{loc} is the current orientation error and e_ω and e_η are defined as for translation. For general rotations, the Baker–Campbell–Hausdorff formula adds terms of second order in the angles. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, has the world position $\mathbf{p} + R\mathbf{c}$, so its error is $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^\top - I)R\mathbf{c}$. The operator norm of $\hat{R}R^\top - I$ for a rotation by θ is $2\sin(\theta/2)$, which gives

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}} - \mathbf{p}\| + 2r\sin(\theta/2) \le \|\hat{\mathbf{p}} - \mathbf{p}\| + r\theta, \tag{S6}$$

which is the contact bound of Sec. 3.1. Orientation errors therefore enter the contact geometry scaled by the lever arm, and a grasp or placement that depends on orientation needs an SE(3) target state. For rotationally symmetric targets, rotation about the symmetry axis leaves every contact unchanged, and we exclude it from inputs, labels, and errors.

G.3. Context and Clearance

Let $\mathcal{P}$ be the set of context points and times that the executed path passes, and suppose that the path keeps every point $\mathbf{x}$ of the robot and the carried object at least m away from the *predicted* context positions $\hat{\mathbf{y}}_t(\mathbf{z}, \tau)$ for $(\mathbf{z}, \tau) \in \mathcal{P}$. A collision requires $\mathbf{x} = \mathbf{y}_t(\mathbf{z}, \tau)$ for some $(\mathbf{z}, \tau) \in \mathcal{P}$, and then $\|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{y}_t(\mathbf{z}, \tau)\| = \|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{x}\| \ge m$, so a collision requires a prediction error of at least m somewhere along the path. Context precision finer than m thus has no first-order effect on success. The bound concerns the size of the error and says nothing about sampling. A flow sampled on a grid misses structures thinner than its spacing and errs at motion boundaries. Both effects enter the path error e_F, which we measure on held-out points along the executed paths (Appendix F.3) instead of inferring it from the grid spacing. The margin is itself a design variable, since a policy

can keep a larger margin when its context estimate is coarse, at some cost in reachability or time.

G.4. Leads Under Rolling Replanning

With chunks of H commands executed in order, one per control period from the time a chunk becomes available, the first command of query t executes after the latency ℓ_t, and further commands execute only until the next chunk replaces them. With replanning after every executed command, the relevant leads are therefore $\Delta_{t,j} = \ell_t + j\Delta_a$ for the few j that execute before the next chunk arrives. For the p95 time-to-action of SE(3)-WAM plus the largest added delay of our sweep (200 ms), these leads stay inside the 400 ms of the forecast, so the forecast losses supervise the representation at every instant at which a command can execute. The action stream does not receive the latency, so the policy does not adapt its commands to a particular delay. Replanning after every command keeps the executed leads short instead, and the latency sweep measures how success degrades as they grow.

G.5. Scaling Relations

Three relations follow from the decomposition. Fig. S6 shows the measurements that relate to the first two.

Critical lead. A policy that does not compensate motion ($\hat{\mathbf{v}} = 0$, $\hat{\boldsymbol{\rho}} = 0$) incurs $\|\mathbf{v}\|\Delta$ in addition to e_{loc} and $\|\boldsymbol{\rho}\|$. With a small localization error and smooth motion, its contacts fail once $\|\mathbf{v}\|\Delta > \varepsilon$, i.e., beyond the critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$, and its success depends on the task, the speed, and the latency only through $\|\mathbf{v}\|\Delta/\varepsilon$. Table S6 lists ε, $\|\mathbf{v}\|$, and $\Delta^\star$ for every task. Fig. S6a plots the success of each method against $\|\mathbf{v}\|\Delta/\varepsilon$, computed per episode at the first command's execution time and pooled over tasks, speeds, and latencies, together with the Spearman correlation and a fitted logistic curve for the reactive policies. The reactive policies [fall on one curve] (Spearman $\rho = [X.XX]$), and their success halves at [X.X]. The decomposition gives only a threshold of about one, not the shape of this curve. The capture tasks scatter most around it, because their success also depends on when the gripper closes.

Pose noise. A least-squares velocity estimate from n equally spaced poses that span T, each with independent noise σ per axis, has the standard deviation $\sigma\sqrt{12(n-1)/(n(n+1))}/T \approx \sigma\sqrt{12/n}/T$ [5]. A pose history therefore keeps $\hat{\Delta}e_v$ below the tolerance up to $\sigma^\star = \varepsilon T\sqrt{n}/(\Delta\sqrt{12})$. For $\varepsilon = 10$ mm, $T = 350$ ms, $n = 8$, and $\Delta = 200$ ms, this gives $\sigma^\star \approx 14$ mm. A learned estimator with a motion prior can do better, so $\sigma^\star$ is the value for a model-free linear estimate. The SE(3) history of SE(3)-WAM falls to the level of the best visual history at [X] mm of injected noise (Fig. S6b), [X.X] times $\sigma^\star$ and [X] times the tracker's error.

Figure placeholder: Fig. S6 — uncompensated motion and pose noise

(a) x: $\|\mathbf{v}\|\Delta/\varepsilon$ at the first command's execution time (log); y: SR. One point per method × family × speed × latency; reactive policies (gray) with a fitted logistic curve and Spearman ρ, the threshold at 1 marked; motion-compensating baselines and SE(3)-WAM (orange) as shifted curves. **(b)** x: injected pose noise σ (mm, symlog); y: SR of the SE(3) history; band: best visual history; vertical markers: $\sigma^\star$ and the tracker's measured error.

Figure S6. **Measured trends.** (a) Success of reactive policies against the uncompensated motion relative to the tolerance. Motion compensation shifts the curve to the right. (b) Value of the SE(3) history under injected pose noise.

Clearance. A context forecast with path error e_F avoids collisions while $m > e_F$. A coarse flow should therefore match finer flows at clearances above its measured e_F and fall behind below it.

G.6. Magnitudes in Dynamic-14

Table S24 evaluates the terms for representative Dynamic-14 settings (Table S7). Each of them is comparable to or larger than the millimeter-scale tolerances of the success conditions (Table S5).

Table S24. Illustrative magnitudes of the terms of Eq. 3 for a 200 ms lead. The values follow from the stated quantities and are not measurements.

Term	Setting	Size
$\|\mathbf{v}\|\Delta$	conveyor at 0.10 m/s	20 mm
$\hat{\Delta}e_v$	10% speed misestimate	2.8 mm
$\|\mathbf{v}\|e_\tau$	50 ms timing error (one period)	7.0 mm
$2r\sin(\theta/2)$	$\omega = 1.2$ rad/s unpredicted, $r = 3$ cm	7.2 mm
e_{dyn}	ball on a 10° ramp, no acceleration model	24 mm
ε	grasp of the conveyor part	[X] mm
grid spacing	14 × 14 flow at the workspace	[X] mm

The ramp entry uses the acceleration of a solid sphere that rolls without slipping, $a = \frac{5}{7}g\sin\alpha \approx 1.2$ m/s^2 at $\alpha = 10°$, and $\frac{1}{2}a\Delta^2$ for $\Delta = 0.2$ s.

G.7. Scope

The decomposition addresses the approach to a rigid target whose motion is exogenous until contact. It does not cover deformable or articulated targets, whose motion field on $\mathcal{O}$ is not determined by six numbers. It also does not cover the motion after contact, which depends on the action itself and

calls for action-conditioned prediction, the case in which letting the world read the action could matter. Scenes in which the target's role changes over time (e.g., sequential multi-object tasks) are outside its scope as well. Perception quality enters only through e_{loc} and e_v, and we measure it separately (Table S1). The bound is not a probability of success. It identifies which errors scale with which factors.